PMaO project

Decision Support System for an automotive manufacturing facility

Team Members: Marco Buonfantino,
Andrea Cevenini

Alma Mater Studiorum - Università di Bologna

Academic Year 2025/2026

Table of contents

Chapter 1: Introduction 3
1.1 Manufacturing Environment and DSS Architecture 3
1.2 Methodological Framework: The Hierarchical Approach 3
1.3 Production Mix 4
Chapter 2: Resource Requirement Planning (RRP) 7
2.1 Scope and Assumptions 7
2.2 Key Parameters and Decision Variables 8
2.3 Objective function 8
2.4 Constraints 9
2.5 Results and Discussion 10
2.6 Model Validation and Stress-Testing 11
Chapter 3: Master Production Schedule (MPS) 14
3.1 Scope and Assumptions 14
3.2 Key Parameters and Decision Variables 15
3.3 Objective Function 16
3.4 Constraints 16
3.5 Results and Discussion 18
3.6 Model Validation and Stress-Testing 20
Chapter 4: Sustainable Material Requirement Planning (Green MRP) 23
4.1 Scope and Assumptions 23
4.2 Key Parameters and Time Phasing (Explosion of the BOM) 24
4.3 Objective Function 25

4.4 Constraints 26
4.5 Results Analysis and Model Verification 27
4.5.1 Mass-Balance and Safety Stock Absorption 28
4.5.2 The "Green" Logistical Switch 30
4.6 Model Validation and Stress-Testing 31
Chapter 5: Operational Job Shop Scheduling 33
5.1 Scope and Physical Assumptions 33
5.2 Mathematical Variables and Decision Architecture 34
5.3 Objective Function 35
5.4 Mathematical Constraints 35
5.5 Model Execution and Results Analysis 37
5.5.1 Operational Translation and Shift Feasibility 37
5.5.2 Technological Routing Validation (StartTime) 37
5.5.3 Parallel Allocation and Spatial Integrity (Alloc) 38
5.6 Scheduling Gantt Chart 39
Chapter 6: Project Conclusion and Framework Synthesis 40
6.1 An Integrated Hierarchical Architecture 40
6.2 Strategic Capacity and Master Scheduling (RRP & MPS) 40
6.3 Sustainable Material Sourcing (Green MRP) 40
6.4 Autonomous Shop-Floor Execution (Job Shop Scheduling) 41
6.5 Final Remarks 41

Chapter 1: Introduction

1.1 Manufacturing Environment and DSS Architecture

This project models and optimizes a manufacturing facility dedicated to producing advanced mechatronic and automotive components (including Brakes, Clutches, Pistons, Shafts, Shifts, and Wheels). These products are characterized by multi-stage production cycles and complex Bills of Materials (BOM).

From a physical layout perspective, the production facility operates as a highly flexible **Job Shop** environment. Unlike rigid flow-shops, this layout is designed to handle a heterogeneous product mix where each item follows a customized routing sequence. The shop floor comprises several distinct work centres. Each work centre is equipped with a specific set of industrial machines and skilled operators, capable of processing multiple product types. The routing of each product through the shop floor is determined by its unique technological requirements, involving dynamic sequences of operations across various departments.

This dynamic environment inherently risks the formation of moving bottlenecks and machine idle times, necessitating an advanced planning architecture. To govern this operational complexity, we developed a comprehensive **Decision Support System (DSS)**. Our architecture relies on a relational **Microsoft Access Database**, which handles all parameters, from routing steps and machine parameters to supplier distances, and feeds the data directly into the mathematical models. The models are coded in **AMPL** and solved via the **Gurobi optimization engine**, ensuring that every operational decision is mathematically proven to minimize costs while strictly respecting physical, temporal, and environmental constraints.

1.2 Methodological Framework: The Hierarchical Approach

An essential architectural decision in the configuration of our Decision Support System involves the temporal and capacity wise structuring of the planning horizons.

Given the combinatorial complexity of the manufacturing environment, this project implements a **Hierarchical Production Planning** approach. The system divides decisions into distinct temporal and aggregation layers, where the outputs of the upper-level models establish the rigid boundary parameters for the lower-level optimizations:

1. **Resource Requirement Planning (RRP):** Operates on an **annual macro horizon**. It aggregates the global market demand against a robust, calendar driven availability matrix (isolating effective working days from holidays). Its objective is purely strategic and structural: sizing the baseline machine park, resolving macro-capacity bottlenecks, and determining the exact absolute number of operators required to run the facility.
2. **Master Production Schedule (MPS):** Disaggregates the annualized strategic targets into monthly buckets over a 12-month rolling horizon. It balances production levelling against inventory holding costs, determining the exact aggregate production volumes (`X[p,t]`) to

be launched each month while respecting manufacturing lead times and preventing stockouts.

3. **Material Requirements Planning (MRP):** Synchronizes the monthly aggregate production blocks with the upstream supply chain. It handles the BOM explosion, coupling the continuous requirements of raw materials (e.g., metal sheets) with the discrete purchasing constraints of the suppliers (e.g., standard boxes), while actively optimizing the logistics network based on distances and CO2 emission taxes (Green Logistics).
4. **Job Shop Scheduling:** Eliminates discrete time buckets entirely, shifting the mathematical landscape into a continuous time paradigm at the minute level. Executed as a short-term operational translation of the MPS grid, it solves localized sequencing, resolves non-overlapping disjunctive resource constraints (Big M), allocates jobs to parallel physical machines, and orchestrates minute by minute worker allocations (incorporating advanced technique for manual assembly).

By establishing this decoupled, multi-level flow, our DSS guarantees highly robust decisions, achieving an optimal trade-off between mathematical solvability, strategic stability, and microscopic operational precision.

1.3 Production Mix

The primary objective of this Decision Support System (DSS) is the strategic and operational management of the manufacturing facility operating within the automotive sector.
The production baseline encompasses six distinct product families.

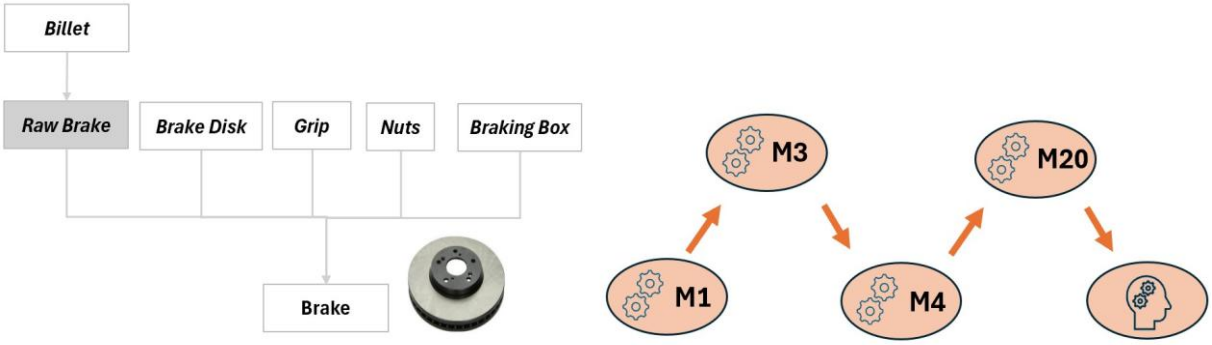


Figure 1.1: Brake BOM Decomposition and machine sequencing

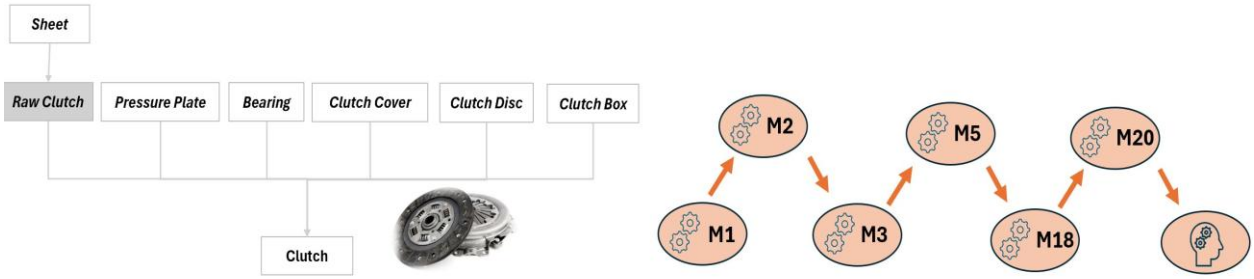


Figure 1.2: Clutch BOM Decomposition and machine sequencing

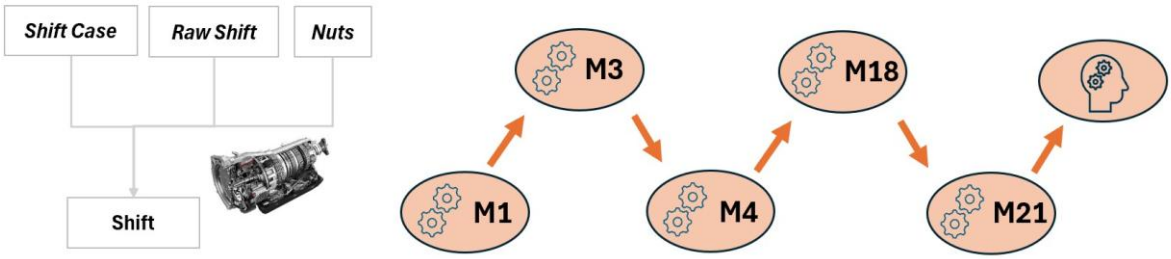


Figure 1.3: Shift BOM Decomposition and machine sequencing

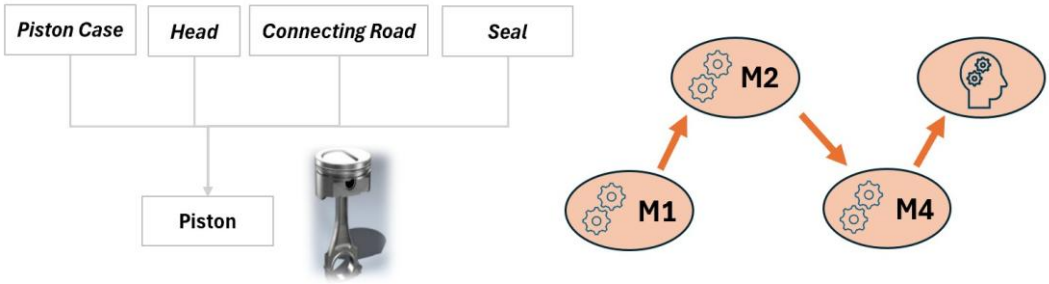


Figure 1.4: Piston BOM Decomposition and machine sequencing

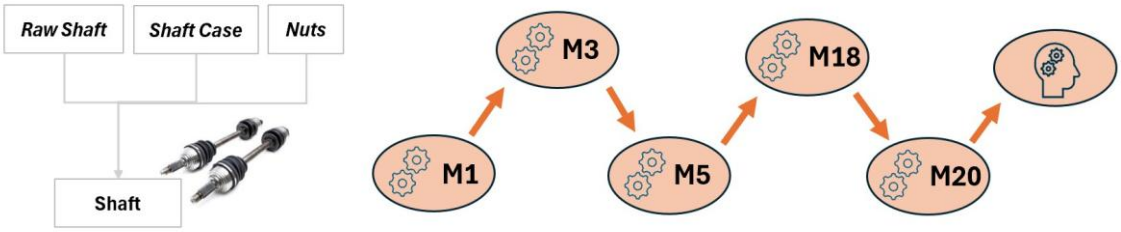


Figure 1.5: Shaft BOM Decomposition and machine sequencing

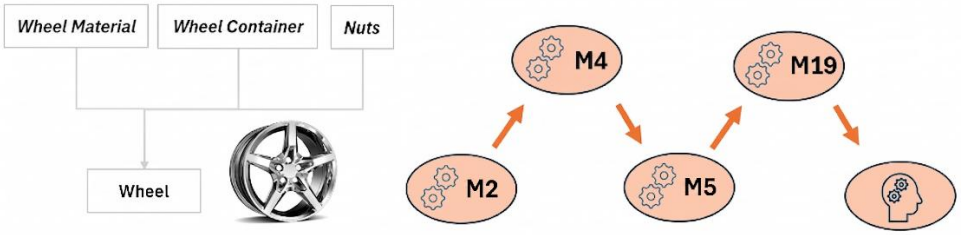


Figure 1.6: Wheel BOM Decomposition and machine sequencing

By modelling the system explicitly through the relational entries of the ProdMach, Product, and Jobs tables present in the Access database, the DSS establishes an immutable data pipeline. Any modification within a component's mechanical footprint or routing precedence automatically triggers a reconfiguration of the constraint within the AMPL mathematical models.

Chapter 2: Resource Requirement Planning (RRP)

2.1 Scope and Assumptions

The RRP model represents the macroscopic level of our planning. The main goal is to define the company's baseline strategy over a long-term planning horizon, assumed to be **one calendar year**. The algorithm must decide which customer orders should be accepted to maximize the profit margin, define the optimal budget for purchasing new CNC machinery, and calculate the exact requirement for seasonal personnel to support the baseline workforce.

To make the model adhere to reality, we made a few fundamental **engineering assumptions**:

- **Annual Horizon and Working Calendar:** To model capacity accurately, a specific table named `CALENDAR` was created within the Access database that maps the actual working days of the factory. This calendar is read in AMPL via the `Lavorativo` parameter (which takes a value of 1 for working days and 0 for holidays), allowing the algorithm to calculate the exact available production days without overestimating the available hours. Consequently, AMPL automatically computes the `Horizon_Days` parameter by summing all the "1" values from this calendar, resulting in a total of 261 effective working days for our specific year.
- **Order Quantity (`OrdQ`) Calculation:** To determine the exact number of items contained in a single order, we created the `OrdQ` column within the `Product` table in the Access database. We calculated this by taking the total annual `Demand` for each product (present in the table `Product` on Access) and dividing it by the sum of all orders placed for that specific product throughout the entire year (tracked by the `DemandProfile` column in the `ORDPROD` Access table). This provides a realistic and precise batch size for every issued order.
- **Financial Definition of Revenue:** We assumed that the `Revenue` parameter (read from the Access `Product` table) does not represent the simple gross selling price. Instead, it is defined as the *Selling Price minus the Cost of Raw Materials*. This ensures the model optimizes based on the actual marginal contribution of each product.
- **Logistics and Machine Parallelism:** We assume that material handling time (`Handling`) only impacts the operator's workload. While the forklift driver moves a piece from one station to another, the CNC machine can be reloaded and continue producing without any interruption.
- **Assembly Isolation:** We assume that final assembly is a purely manual operation. The workbenches are not bounded by the shifting or efficiency logic typical of machine tools, therefore the `ManualAssembly` time will impact only the workers time and not the machine constraint.
- **Dual Impact of Setup Time:** As it happens on the real shop floor, the `SetupTime` is actively considered in ***both*** **capacity constraints**. Preparing a machine for a new product batch requires the machinery to be completely stopped (thus consuming available machine

hours) and simultaneously requires an operator to physically equip it with the necessary tools (thus consuming available labour hours).

- **Standard Shifts:** We assume a standard working shift of 8 hours per day for calculating both human and machine capacity.
- **Zero Baseline Workforce (`wt = 0`):** To accurately determine the absolute workforce requirement from scratch, we assumed the initial baseline workforce (`wt`) is zero. Consequently, the model uses the variable `sw` to calculate the *total* number of operators the company needs to hire to run the entire production plan, acting as a pure sizing tool rather than just calculating a seasonal surplus.
- **Pre-existing Assembly Infrastructure:** It is assumed that the manual assembly benches (`AssemblyTable`) are pre-existing physical assets already available on the shop floor. Consequently, they do not require any new capital expenditure and must not be evaluated as potential bottlenecks in the automated machine capacity calculations. To natively enforce this logic within the AMPL architecture, we engineered specific filtered subsets (`RRP_MACHINES` and `RRP_PRODMACH`) by utilizing the set difference operator (`diff`). This ensures that the optimization algorithm calculates capacity limits and depreciation quotas solely on the actual CNC processing machinery, while the operational throughput of the final assembly phase is strictly dictated by the volume of the workforce hired.
- **Adequate Facility Footprint:** It is assumed that the current plant layout possesses sufficient pre-allocated expansion areas to accommodate the necessary volume of new machinery evaluated by the RRP model. Consequently, floor space is considered a non-binding constraint for the scope of this capacity planning.

2.2 Key Parameters and Decision Variables

The model relies on three key decision variables that shape the factory layout:

- `Accepted`: A binary variable acting as a logical switch. It is equal to 1 if a specific product order in a given month is accepted, and 0 if it is rejected.
- `MachQuantity`: An integer variable indicating the number of physical machines of a specific type to be purchased and installed on the shop floor.
- `sw` (Seasonal Workers): An integer variable representing the number of seasonal workers to hire.

These decisions are made by balancing economic parameters (the aforementioned `Revenue`, the machine purchase `CapitalCost`, the personnel cost `wc`) with operational parameters (such as the processing time `ProcTime`, the `SetupTime`, and the `ManualAssembly` times).

2.3 Objective function

The compass guiding the entire RRP model is the maximization of the corporate net profit (`Production_Profit`). This mathematical formulation captures the complete economic footprint of the manufacturing plant. Gurobi evaluates millions of operational and structural scenarios to find the exact configuration that maximizes the following equation:

```
#OBJECTIVE FUNCTION
maximize Production_Profit:
    # 1.Total sales revenue
    sum {(p,t) in ORDPROD} (Revenue[p] * DemandProfile[p,t] * OrdQ[p] * Accepted[p,t])

    # 2. Production Operating Costs calculated only on real routings
    - sum {(p,t) in ORDPROD, m in RRP_MACHINES: (p,m) in PRODMACH}
      ((ProcTime[p,m] * DemandProfile[p,t] * OrdQ[p] * Accepted[p,t]) * ProductionCost[m])

    # 3. Fixed Capital Expenditures for Machinery Depreciation
    - sum {m in RRP_MACHINES} (((CapitalCost[m]*1000 * Horizon_Days) / (Lifespan[m]*365)) * MachQuantity[m])

    # 4. Penalty costs
    - wc * sw;
```

Specifically, the objective function calculates the total sales revenue generated by the accepted demand profiles, and strictly subtracts three fundamental cost pillars:

- **Operational Expenditures:** The variable production costs strictly tied to the active processing time (`ProcTime`) of the CNC machines. This ensures the solver mathematically penalizes physically inefficient technological routings.
- **Capital Expenditures:** The depreciation quotas for the newly purchased physical machinery (`MachQuantity`), scaled dynamically according to the actual duration of the planning horizon (`Horizon_Days`).
- **Labor Penalty Costs:** The structural economic impact of hiring the required seasonal workforce (`sw`) to sustain the physical capacity, manage inter-department logistics, and properly execute the frequent machine setups on the shop floor.

2.4 Constraints

To ensure the strategic plan is physically feasible on the shop floor, the objective function is strictly bound by two capacity constraints, which manage two quite different natures of our production system:

- **Machine Capacity Constraint (Machine_Capacity):** This fundamental constraint ensures that the total processing and setup times required by the accepted orders within a specific period do not exceed the actual machine hours available during that same time window. To prevent the creation of unfeasible production plans, the mathematical formulation evaluates the capacity on a strict month-by-month basis (`t in PERIODS`). The available capacity (the right side of the inequality) is dynamically calculated by multiplying the decision variable representing the purchased machines (`MachQuantity`) by the daily shifts, the standard 8 working hours (converted into minutes), the specific machine efficiency (`DeltaTj`), and the exact number of actual working days in that specific month (`Period_Days[t]`). On the workload side, the equation rigorously multiplies the `SetupTime` by the number of individual orders (`DemandProfile`), ensuring that the physical time required for machine changeovers is precisely accounted for every single time a new batch (or order) is processed.

```
#Machine constraint evaluated month per month
subject to Machine_Capacity {m in RRP_MACHINES, t in PERIODS}:
    sum {p in PRODUCTS: (p,t) in ORDPROD and (p,m) in RRP_PRODMACH}
        ( (ProcTime[p,m] * OrdQ[p] + SetupTime[p,m]) * DemandProfile[p,t] * Accepted[p,t] )
    <= MachQuantity[m] * (Shift_Day[m] * 60 * 8 * DeltaTj[m] * Period_Days[t]);
```

- **Labor Capacity Constraint (Labor_Capacity):** This constraint ensures that the total time required for all manual operations is entirely absorbed by the available workforce on a strict month-by-month basis (`t in PERIODS`). To guarantee a physically feasible shop floor, the equation explicitly computes the exact labour footprint by summing two distinct macro-activities. The first component calculates the time required for the final manual assembly (`ManualAssembly`) and the ultimate handling step to transport the pieces to the workbenches. The second component accounts for the inter-machine cart logistics (`Handling`) and the changeover activities (`SetupTime`), which are rigorously multiplied by the number of individual orders (`DemandProfile`) to reflect the true operational effort required for frequent batch switching. By comparing this total monthly workload against the actual human capacity, derived from the baseline workers (`wt`, which is set to zero), the newly hired seasonal staff (`sw`), and the precise working days of that specific month (`Period_Days[t]`), the optimization model sizes the workforce exactly to sustain peak production periods without bottlenecking the CNC machinery.

```
#Workers constraint evaluated month per month
subject to Labor_Capacity {t in PERIODS}:
    #1.Calculation of work at the assembly bench + 1 Final handling to bring the pieces
    #from the last machine to the assembly bench
    sum {p in PRODUCTS: (p,t) in ORDPROD}
        ( (ManualAssembly[p] + Handling[p]) * DemandProfile[p,t] * OrdQ[p] * Accepted[p,t] )
    +
    # 2. Work calculation for CNC machines (Handling between machines + Setup for EACH order)
    sum {p in PRODUCTS, m in RRP_MACHINES: (p,t) in ORDPROD and (p,m) in RRP_PRODMACH}
        ( (Handling[p] * OrdQ[p] + SetupTime[p,m]) * DemandProfile[p,t] * Accepted[p,t] )
    <= (wt + sw) * 8 * 60 * Period_Days[t];
```

2.5 Results and Discussion

```
Horizon_Days = 261

Production_Profit = 11307600

MachQuantity [*] :=
AssemblyTable  0
          M01  3
          M02  3
          M03  2
          M04  1
          M05  2
          M18  1
          M19  1
          M20  2
          M21  1
;

Accepted [*,*] (tr)
:  Brake Clutch Piston Shaft Shift Wheel    :=
1     1     1      1     1     1     1
2     1     1      1     1     1     1
3     1     1      1     1     1     1
4     1     1      0     1     1     1
5     1     1      1     1     1     1
6     1     1      1     1     1     1
7     1     1      1     1     1     1
8     1     1      1     1     1     1
9     1     1      0     1     1     1
10    1     1      1     1     1     1
11    1     1      1     1     1     1
12    1     1      1     1     1     1
;

sw = 61
```

The results confirm the high profitability of the production plan and validate the engineering solidity of the mathematical model. Over a calculated horizon of 261 actual working days (`Horizon_Days`), the system generates a total net profit of €11,307,600 (`Production_Profit`).

While the operating margins associated with these components structurally exceed the depreciation and hiring costs driving the optimizer to accept most of the order portfolio the model demonstrates crucial tactical intelligence. As shown by the results matrix, the `Accepted` variable equals 1 for almost all product families across the year. However, the solver strategically rejects the orders for the Piston during periods 4 and 9 (`Accepted['Piston', 4] = 0` and `Accepted['Piston', 9] = 0`). This specific rejection proves that the strict month-by-month capacity constraints act effectively as an operational shield,

preventing the facility from accepting unfeasible workloads during seasonal demand peaks.

To sustain this optimized production volume without saturating the facility, the solver outlined a balanced layout between capital investments and labour operational costs:

- **Machine Purchasing (MachQuantity):** The solver established the activation of **16 total physical units** in the CNC machining department. The algorithm tripled the capacity (3 units) for the Welding stations (M01) and Drilling stations (M02), identifying them as the primary bottlenecks heavily stressed by the products' technological routings and frequent setups. Furthermore, it doubled the capacity (2 units) for the Lathing (M03), M05, and M20 workstations. For the remaining machine types (M04, M18, M19, M21), a single unit proved perfectly sufficient to cover the requirements. Finally, the optimization correctly allocated 0 units to the `AssemblyTable`, mathematically validating the assumption defined in the initial scope: since the manual benches are pre-existing assets, the model successfully isolated them from the purchasing budget.
- **Workforce Sizing (sw):** The optimization quantified the personnel requirement to run the entire factory at full capacity at **61 total operators**. Having set the initial permanent workforce to zero (`wt = 0`), this number represents the real and absolute dimension of the required shop floor team. This pool of 61 people constitutes the exact mathematical equilibrium point capable of absorbing the dense, simultaneous load of frequent machine changeovers (`SetupTime`), internal cart logistics (`Handling`), and final manual assembly. This sizing guarantees that no CNC machine sits idle waiting for an operator to perform a setup, ensuring that every accepted order is physically completed within its designated monthly time bucket.

2.6 Model Validation and Stress-Testing

As demonstrated in the previous section, the baseline RRP optimization already exhibits tactical intelligence by rejecting the Piston orders during peak bottleneck periods (`Accepted['Piston', 4] = 0` and `Accepted['Piston', 9] = 0`), proving it does not show a 100% Service Level.

To further validate the elasticity of our Decision Support System (DSS) and observe how the optimal baseline reconfiguration adapts to extreme industrial conditions, we stress-tested the mathematical model under three severe simulated shocks:

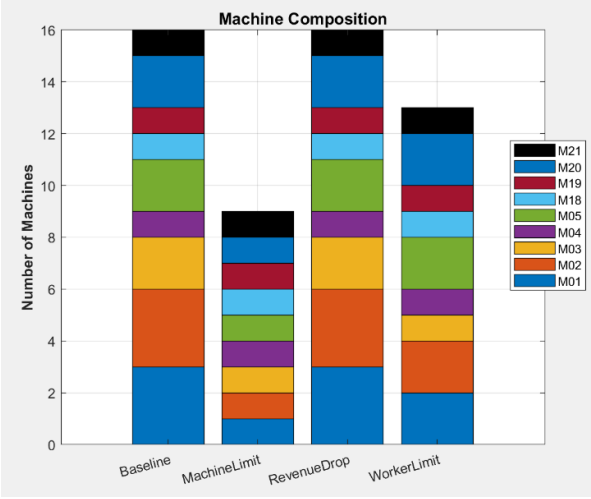
- **Scenario A: Floor Space or Budget Limits:** We constrained the maximum number of purchasable machines per category (`MachQuantity[m] <= Limit`), simulating a strict corporate capital budget or absolute physical floor space limitation within the facility. Once the absolute physical capacity limit of the machines was breached, the system did not crash into an "Infeasible" mathematical state. Instead, the DSS strategically dropped additional specific orders, beyond the baseline Pistons, which consumed the highest machine minutes per euro of profit, proving its capability to maximize returns even under severe financial and spatial constraints.

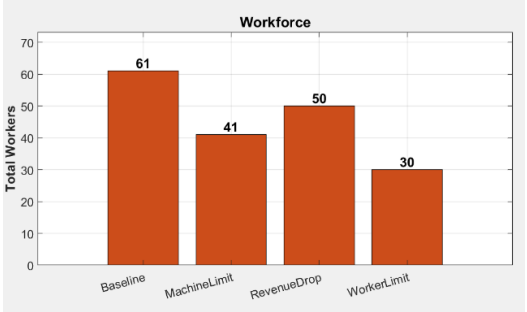
- **Scenario B: Economic Shock (Revenue drop of a product):** We simulated a severe market shock by drastically reducing the `Revenue` parameter for a specific product family. As the profit margin collapsed, the product became mathematically inconvenient to

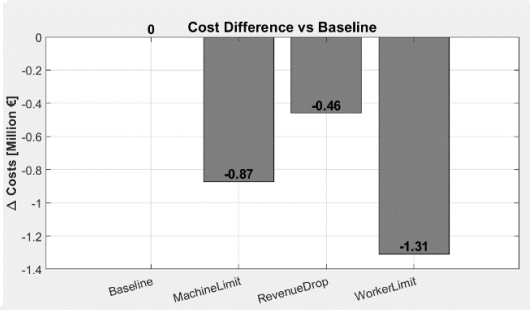
produce. Instead of blindly fulfilling the demand and absorbing financial losses to keep the service level, the DSS autonomously triggered its safety valve. The solver flipped the binary acceptance variable to zero (`Accepted[p,t] = 0`) specifically for the degraded product, freeing up machine and labour capacity to prioritize higher-margin items.

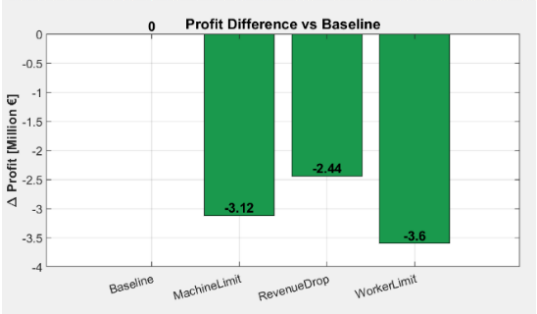
- **Scenario C: Labor Shortage:** To evaluate the balance between Capital and Operational Expenditures, we introduced a strict upper bound on seasonal hiring (`sw <= Limit`), simulating a regional shortage of qualified manual workers. Faced with a highly restricted labour pool, the system dynamically reconfigured its operational matrix. Since the solver could no longer hire its way out of the setup and assembly bottlenecks, it reacted by dropping high-volume orders characterized by intensive `ManualAssembly` and `Handling` times. It prioritized exclusively those orders relying primarily on automated `ProcTime`, effectively diluting the scarce labour resource over a wider, highly automated asset base.

Comparisons: baseline vs model variations









Making the comparison between the 3 variations we do not see any very critical problem or very big drop on profit. The solver finds a good production mix to minimize the difference in revenue and costs. This comparison helps us to validate the model and by changing some critical parameters, we obtained behaviours that make sense. The substantial difference is, as expected, in the acceptance of the different orders month by month.

In scenario A we set a limit of space of 9 machine in total. Therefore, the solver prioritizes the product with lower machine time, without any constrain on worker labour time.	In scenario B we set the brake revenue dropped by 90%. The solver prioritizes other order, but if the machines are free from other product Gurobi can accept the Brake order anyway.	In scenario C we set a limit of 30 workers. In this case the solver prioritizes products with lower worker labour time needed, preferring product with higher machine time.
Accepted [*,*] (tr) : Brake Clutch Piston Shaft Shift Wheel 1 1 0 0 0 1 1 2 1 1 0 0 1 1 3 1 1 0 0 1 1 4 1 0 0 0 1 1 5 1 0 1 0 1 1 6 1 0 1 0 1 1 7 1 0 0 0 1 1 8 1 1 1 0 1 1 9 1 1 1 0 1 0 10 1 0 0 0 1 1 11 1 0 1 0 1 1 12 1 0 0 0 1 1	Accepted [*,*] (tr) : Brake Clutch Piston Shaft Shift Wheel 1 0 1 1 1 1 1 2 1 1 1 1 1 1 3 0 1 1 1 1 1 4 0 1 0 1 1 1 5 1 1 1 1 1 1 6 0 1 1 1 1 1 7 0 1 1 1 1 1 8 1 1 1 1 1 1 9 0 1 1 1 1 1 10 0 1 1 1 1 1 11 1 1 1 1 1 1 12 0 1 1 1 1 1	Accepted [*,*] (tr) : Brake Clutch Piston Shaft Shift Wheel 1 1 1 0 0 1 0 2 1 0 0 0 1 1 3 0 1 0 0 1 1 4 1 1 0 0 0 1 5 1 1 1 1 0 1 6 0 1 1 0 1 1 7 1 1 1 0 0 1 8 1 1 0 1 1 1 9 1 0 0 0 0 1 10 1 1 0 0 0 1 11 1 1 0 1 0 1 12 0 1 0 1 0 1

Chapter 3: Master Production Schedule (MPS)

3.1 Scope and Assumptions

While the RRP sets the macro-strategic boundaries over an entire year, the Master Production Schedule (MPS) operates at the tactical level. Its scope is to disaggregate the total annual volume into discrete **monthly time periods** from January to December (`PERIODS = 1…12`). The MPS must decide exactly ***when* to produce and *how much* to produce**, seeking the absolute minimum total cost by balancing the expense of storing inventory against the cost of initiating production batches.

To accurately reflect our Job Shop reality, the mathematical formulation is built upon several key engineering assumptions:

- **Lead Time Variability (`LTPP`):** We assume a standard production lead time of 1 month for most mechatronic components (Brake, Clutch, Piston, Shaft, Wheel). This means an order launched at the beginning of the month will be physically completed and available at the end of the month. Conversely, we assumed that the 'Shift' component has a lead time of 0, meaning its manufacturing cycle is short enough to be launched and fully completed within the very same monthly period. These data can be found on the column `LTPP` in the table `Product` on the Access database.
- **Strategic Initial Inventory (`InitInv`):** This assumption is strictly tied to the lead time. Since products with a 1-month lead time cannot physically satisfy January's demand through January's new production, we must rely on stock carried over from the previous year. Therefore, the `InitInv` parameter in the Access database is deliberately set to hold the quantity of products necessary to fulfil the demand of the first month, preventing an unavoidable initial stockout.
- **Maximum Batch Capacity (`MaxBatchCap`):** We assume that production batches have strict physical limitations (e.g., pallet capacity or oven space). A single batch cannot hold an infinite number of products. The `MaxBatchCap` parameter acts as a hard threshold: once the required production quantity exceeds this limit, the solver is forced to trigger the opening of a new batch (`SPO + 1`), so incurring an additional fixed batch cost.
- **Minimum Production Threshold (`MinBatchProd`):** To avoid severe economic inefficiencies, we assume the factory does not produce negligible quantities. It is completely illogical to turn on a machine, equip it, and pay the full setup penalty just to manufacture a single unit (e.g., one brake). Therefore, production is only authorized if the requested volume meets or exceeds the `MinBatchProd` limit, justifying the setup effort.
- **Finite Capacity Planning:** Unlike basic MPS models that assume infinite resources, our model is strictly constrained by the actual plant capacity. We import the physical layout generated by the RRP (specifically, the `MachQuantity` of active machines) and cross-reference it with a real industrial calendar to compute the true available machine hours month by month.
- **Assembly Exclusion Filter:** Consistent with the RRP logic, the assembly tables (`AssemblyTable`) are mathematically excluded from the machine capacity sets using a

software filter (`diff`). This prevents the solver from generating false bottlenecks on operations that depend purely on the human workforce.

- **Cost-Driven Inventory Control:** Instead of imposing a hard physical limit on warehouse volume, we assume the facility possesses sufficient storage space to accommodate our standard product inventory volumes. Consequently, the accumulation of stock is not constrained by spatial limitations but is strictly regulated and penalized by the economic `HoldingCost`, which forces the model to keep inventory levels as lean as possible to avoid tying up capital.
- **Fresh Planning Horizon:** Pre-existing scheduled receipts or promised orders inherited from previous years are assumed to be zero. The factory starts from a clean operational slate.

3.2 Key Parameters and Decision Variables

The transition from a continuous volume target (RRP) to discrete monthly planning (MPS) requires a new set of dedicated decision variables:

- `X[p,t]`: An integer variable representing the physical quantity of product *p* completely manufactured and ready for storage or sale at period *t*.
- `S[p,t]`: An integer variable tracking the inventory level (stock on hand) at the end of period *t*.
- `RO[p,t]`: The number of Production Orders Released (batches) launched into the shop floor at period *t*.
- `SPO[p,t]`: The number of Scheduled Production Orders (batches) fully completed and exiting the machines at period *t*.

These variables interact with two main parameter categories imported from our Microsoft Access Database. Firstly, we have **cost parameters** that include the pure manufacturing expense (`UnitProdCost`), the penalty for keeping products in the warehouse (`HoldingCost`), and the fixed logistical costs of producing a batch (e.g., forklift handling and end-of-line testing), defined as `BatchCost`. Since the problem does not define `UnitProdCost, HoldingCost` and `SetupCost` for each product, then these parameters have been calculated by us in the file **Costs.mod** on AMPL. Then we have the **operational parameters** that include the exact items required per customer order (`OrdQ`), manufacturing lead times (`LTPP`), and the strict physical limits of a batch (`MaxBatchCap` and `MinBatchProd`). Crucially, the demand parameter (`RealDemand`) is not forecasted but is deterministically imported as the *Accepted* demand, which is the output from our previous RRP layer, guaranteeing total architectural consistency; this linking is made in the file MPS.run.

3.3 Objective Function

Since the strategic profit maximization was already resolved by the upper-level RRP model, the MPS shifts its focus entirely toward operational efficiency. The objective is to fulfil the deterministic `RealDemand` at the absolute **minimum total cost**. The solver evaluates the trade-off between manufacturing in massive quantities (which saturates the warehouse and spikes inventory costs) and manufacturing small, frequent batches (which increases setup and handling time and consequently costs). The objective function minimizes the sum of four cost drivers:

```
#OBJECTIVE FUNCTION
minimize Total_Cost:
    sum {p in PRODUCTS, t in PERIODS}
        (UnitProdCost[p] * X[p,t] +
         HoldingCost[p] * S[p,t] +
         SetupCost[p] * RO[p,t] +
         BatchCost[p] * SPO[p,t]);
```

Notice that the fixed `SetupCost` is multiplied by the orders released `(RO)`, charging the penalty at the exact moment the machine is switched on and reconfigured. Conversely, the `BatchCost` (which includes end-of-line logistics and testing) is tied to the scheduled production orders (`SPO`), charging the system only when the batch is physically completed.

3.4 Constraints

To ensure the mathematical plan is physically executable on the shop floor, the cost minimization is bounded by a rigorous set of constraints governing inventory flow, temporal lead times, logical switches, and machine capacity.

- **Inventory Flow Balance (`Inv_Balance`):** This is the fundamental equation linking consecutive time periods. It dictates that the physical inventory carried over from the previous month (`S[p, t-1]`) plus the new production **completed** in the current month (`X[p,t]`) must be exactly equal the market demand required in the current month (`RealDemand[p,t]*OrdQ[p]`) plus the inventory kept for the future (`S[p,t]`).

```
subject to Inv_Balance {p in PRODUCTS, t in PERIODS}:
    S[p, t-1] + X[p,t] = (RealDemand[p,t]*OrdQ[p]) + S[p,t];
```

- **Lead Time (`LeadTime_Link`):** This constraint establishes the direct temporal link between the start and the completion of production. It dictates that the batches declared as "completed" (`SPO[p,t]`) in month t must correspond exactly to those "released" (`RO[p, t - LTPP[p]]`) in the past, specifically in month `t - LTPP` and this is valid only when `t > LTPP`. From an operational standpoint, if a component such as a brake requires a 1-month Lead Time and the batch is launched in January, the finished products will be physically available and ready only in February.

```
subject to LeadTime_Link {p in PRODUCTS, t in PERIODS: t > LTPP[p]}:
    SPO[p,t] = RO[p, t - LTPP[p]];
```

- **Pipeline Initialization Constraint (`Init_Pipeline`):** This constraint defines the boundary conditions for the plant's startup, preventing the model from generating mathematically valid but physically impossible solutions. If the order is released (`RO`) in January (month 1) and the production of that specific batch requires a 1-month lead time, it is physically impossible to record completed batches (`SPO`) during the months of January, as there has not been the necessary physical time to process them. This constraint imposes a restriction on the solver on each month of the year when $t \leq LTPP$, forcing the output to zero until the required Lead Time has entirely elapsed, under the assumption that there is no work-in-progress inherited from the past.

```
subject to Init_Pipeline {p in PRODUCTS, t in PERIODS: t <= LTPP[p]}:
    SPO[p,t] = 0;
```

- **Initial Inventory Constraint (`Init_Inventory`):** This constraint establishes the baseline stock levels at the exact beginning of the planning horizon. It anchors the inventory at period zero (`S[p,0]`) to the specific `InitInv[p]` parameter imported from the Access database, which, as established in the assumptions, represents the physical stock carried over from the previous year to safely fulfil January's demand.

```
subject to Init_Inventory {p in PRODUCTS}:
    S[p, 0] = InitInv[p];
```

- **Setup Activation via Big-M Method (`Setup_Activation`):** To correctly charge the operational costs, we implemented a Big-M logical switch. This constraint forces the solver's hand: if even a single physical piece is yielded into the warehouse (or sold) at time t ($X > 0$), the completion variable `SPO` is forced to be ≥ 1. This directly triggers the `BatchCost` penalty in the objective function for the current period. Furthermore, due to the temporal linkage of the Lead Time constraint (`SPO[p,t]= RO[p, t - LTPP[p]]`), this switch guarantees that the system cannot produce finished goods unless a corresponding `SetupCost` was legitimately incurred and paid at the launch of the cycle. This prevents the system from generating products "for free."

```
subject to Setup_Activation {p in PRODUCTS, t in PERIODS}:
    X[p,t] <= BigM * SPO[p,t];
```

- **Batch Sizing Logic (`Maximum_Batch` & `Min_Production_Threshold`):** The system respects strict physical container limits. If the production volume `X` exceeds the `MaxBatchCap`, the solver is mathematically forced to increment `SPO`, opening an additional batch and paying a penalty. Conversely, the minimum production threshold constraint ensures that the machines are not engaged for negligible quantities: production is completely halted if the required volume does not justify the minimum batch size (`MinBatchProd`), saving the setup effort.

```
subject to Maximum_Batch {p in PRODUCTS, t in PERIODS}:
    X[p,t] <= MaxBatchCap[p] * SPO[p,t];

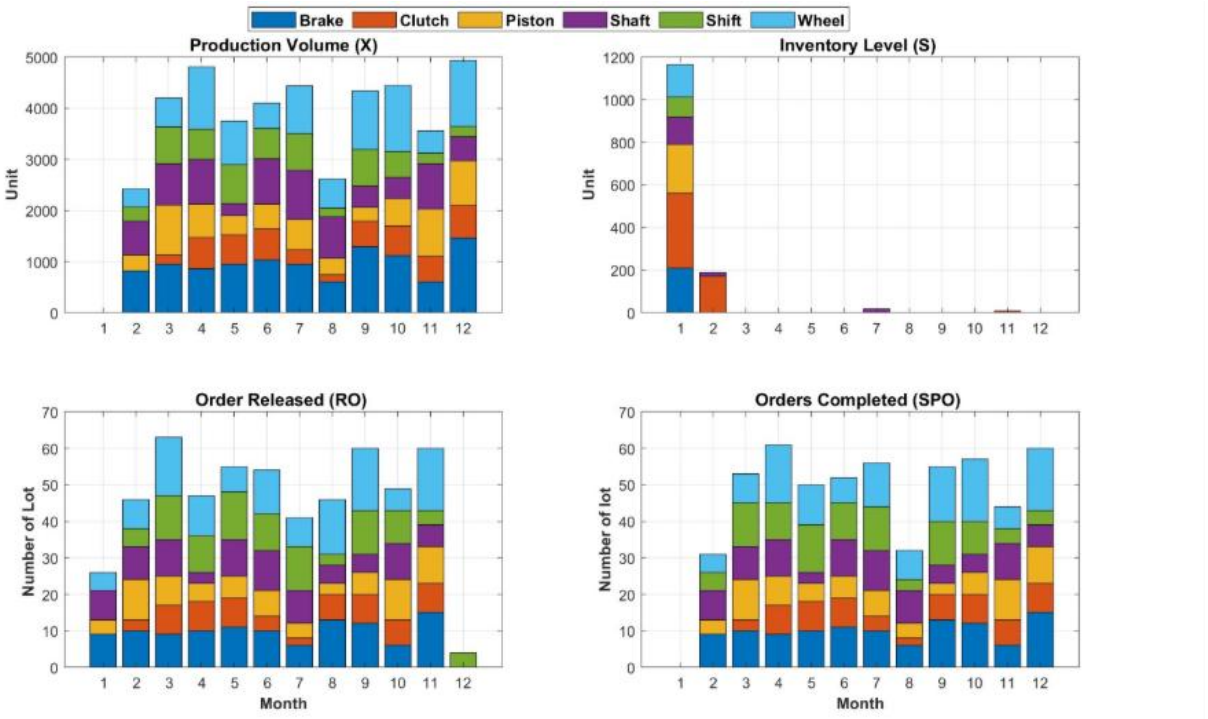
subject to Min_Production_Threshold {p in PRODUCTS, t in PERIODS}:
    X[p,t] >= MinBatchProd[p] * (if SPO[p,t] >= 1 then 1 else 0);
```

- **Temporal Capacity Consumption (`Prod_Capacity`):** This constraint ensures that the workload never exceeds the available monthly machine hours (`AvailCap`). A crucial engineering detail in this formulation is *when* the capacity is consumed. Processing time (`ProcTime`) consumes capacity based on the physical pieces yielded (`X`), whereas the setup time (`SetupTime`) consumes capacity based on the *released* orders (`RO`). This accurately mirrors reality: the operator physically dismantles and equips the machine at the beginning of the cycle, consuming resources at the launch phase, not at the completion phase.

```
subject to Prod_Capacity {m in MPS_MACHINES, t in PERIODS}:
    sum {p in PRODUCTS: (p,m) in MPS_PRODMACH}
    (ProcTime[p,m] * X[p,t] + SetupTime[p,m] * RO[p,t]) <= AvailCap[m,t];
```

3.5 Results and Discussion

The execution of the MPS mathematical model confirms the seamless integration between the tactical capacity planning (RRP) and the operational shop floor scheduling. By inheriting the optimized machine park, the sized workforce, and the strategically filtered order portfolio (`RealDemand`), the Gurobi solver successfully generated a strictly feasible production plan over the 12-month horizon.



The optimization engine converged on a mathematically guaranteed global minimum, yielding a total operational expense of **€273,711** (`Total_MPS_Cost`). This economic figure is the cornerstone of the scheduling algorithm: it represents the absolute optimal balance between competing shop-floor financial drivers. It is the lowest possible combined expense of variable production (`UnitProdCost`), inventory holding penalties (`HoldingCost`), physical machine reconfigurations (`SetupCost`), and batch handling operations (`BatchCost`) required to strictly fulfil the accepted annual demand without violating any physical or temporal constraints.

```
Total_MPS_Cost = 273711
```

```
X [*,*] (tr)
:   Brake Clutch Piston Shaft Shift  Wheel    :=
1       0     0      0     0     0      0
2     822     0    312   655   282    352
3     946   188    972   810   714    576
4     860   612      0   885   588   1224
5     946   576    378   236   756    864
6    1032   612    486   885   588    504
7     948   288    594   960   714    936
8     600   144    324   810   168    576
9    1290   504      0   413   714   1152
10   1120   576    540   413   504   1296
11    600   512    918   885   210    432
12   1462   640    864   472   210   1296
;

S [*,*] (tr)
:   Brake Clutch Piston Shaft Shift Wheel    :=
0    1500   1000   1200   600   600   800
1     210    352    228   128    96   152
2       0    172      0    16     0     0
3       0      0      0     0     0     0
4       0      0      0     0     0     0
5       0      0      0     0     0     0
6       0      0      0     0     0     0
7       2      0      0    16     0     0
8       0      0      0     0     0     0
9       0      0      0     0     0     0
10      2      0      0     0     0     0
11      0      8      0     0     0     0
12      0      0      0     0     0     0
;
```

```
RO [*,*] (tr)
:  Brake Clutch Piston Shaft Shift Wheel    :=
1     9     0      4     8     0     5
2    10     3     11     9     5     8
3     9     8      0    10    12    16
4    10     8      5     3    10    11
5    11     8      6    10    13     7
6    10     4      7    11    10    12
7     6     2      4     9    12     8
8    13     7      0     5     3    15
9    12     8      6     5    12    17
10    6     7     11    10     9     6
11   15     8     10     6     4    17
12    0     0      0     0     4     0
;

SPO [*,*] (tr)
:  Brake Clutch Piston Shaft Shift Wheel    :=
1     0     0      0     0     0     0
2     9     0      4     8     5     5
3    10     3     11     9    12     8
4     9     8      0    10    10    16
5    10     8      5     3    13    11
6    11     8      6    10    10     7
7    10     4      7    11    12    12
8     6     2      4     9     3     8
9    13     7      0     5    12    15
10   12     8      6     5     9    17
11    6     7     11    10     4     6
12   15     8     10     6     4    17
;
```

• **Lead Time Validation and Pipeline Initialization:** The solver's behaviour in the first period (t = 1) perfectly demonstrates the strict adherence to the `Init_Pipeline` constraint and the overall cost-minimization logic. For mechatronic components with a 1-month lead time (Brake, Clutch, Piston, Shaft, Wheel), the physical production matrix (`X`) and the completed batches matrix (`SPO`) register exactly 0 in the first month. The system mathematically recognizes that it cannot complete these items without a prior launch. Crucially, even for the "Shift" component, which is characterized by a 0-month lead time and could theoretically be produced immediately, the solver strategically registers 0 completed batches in January, preferring to bypass the `SetupCost` and `BatchCost` entirely by relying on the initial inventory.

• **Initial Inventory Depletion:** To survive the first month without any production output across all product families, the system successfully relies on the pre-existing warehouse stock. The inventory matrix (`S`) shows the `InitInv` parameters correctly anchored at t = 0 (e.g., 1500 Brakes, 1000 Clutches etc, set on the Access database). During month 1, this stock is consumed to satisfy the immediate demand, dropping to lower quantity levels (e.g., 210 Brakes, 352 Clutches) just in time to cover the sales until the new production batches physically exit the machines in month 2.

• **Batch Synchronization (RO vs. SPO):** The temporal shift dictated by the `LeadTime_Link` constraint is explicitly visible by comparing the order release (`RO`) and order completion (`SPO`) matrices. For a standard component like the Brake, the solver launches 9 batches in month 1 (`RO['Brake', 1] = 9`). Looking at the completion matrix, those exact 9 batches are correctly registered as completed in month 2 (`SPO['Brake', 2] = 9`), perfectly mirroring the physical delay required by the machining routing.

• **End-of-Horizon Lean Behaviour:** As the planning horizon approaches its end, the optimization algorithm perfectly executes a working capital minimization strategy. Recognizing that no demand exists beyond month 12, the solver dynamically stops launching new production for 1-month lead time components in December (`RO = 0` for Brake, Clutch, Piston, Shaft, and Wheel at t = 12). Conversely, since the "Shift" component requires a 0-month lead time, the system accurately registers 4 released orders in December (`RO['Shift', 12] = 4`), which are immediately processed into 4 completed batches (`SPO['Shift', 12] = 4`) within the very same period. Most importantly, the ending inventory matrix at period 12 (`S[p,12]`) is mathematically driven to 0 for all product families. The system produces precisely what is needed to fulfill the final orders, avoiding any unjustified `HoldingCost` penalty for leftover stock.

3.6 Model Validation and Stress-Testing

To establish the structural validity and mathematical robustness of our operational schedule, the MPS model was subjected to a comprehensive sensitivity analysis. To observe how the model handles severe industrial pressures, we tested its elasticity under three extreme operational scenarios, pushing our optimization framework to its limits.

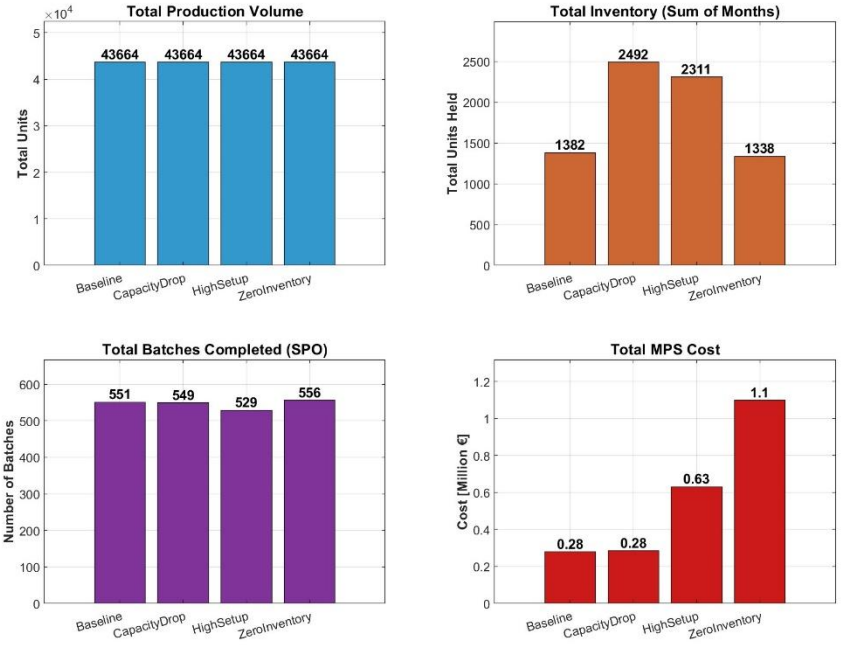
- **Scenario A: Extreme Lean Manufacturing (Holding Cost Inflation):** To observe the inventory dampening effect, we artificially inflated the unit holding cost parameter (`HoldingCost[p]`) to extreme levels, heavily penalizing any stored volume. The solver responded by systematically dismantling the production smoothing profile. Refusing to carry expensive stock, the system shifted toward a pure Just-In-Time (JIT) framework. It fragmented the manufacturing schedule, choosing to pay frequent setup and batch penalties (`SetupCost[p]` and `BatchCost[p]`) to launch discrete micro-batches exactly when needed to satisfy the immediate demand, thereby driving the ending inventory levels (`S[p,t]`) effectively to zero.
- **Scenario B: Setup Cost Surge:** Conversely, we simulated an operational environment where machine reconfigurations and changeovers become drastically expensive by heavily inflating the fixed setup penalty (`SetupCost[p]`). Faced with massive switching costs, the solver aggregated production over time. It launched consolidated batches up to the strict physical threshold dictated by `MaxBatchCap[p]`. The system intentionally chose to over-

produce in a single period, willingly carrying heavy inventory (`S[p,t]`) across several subsequent months to avoid triggering additional setup fees.

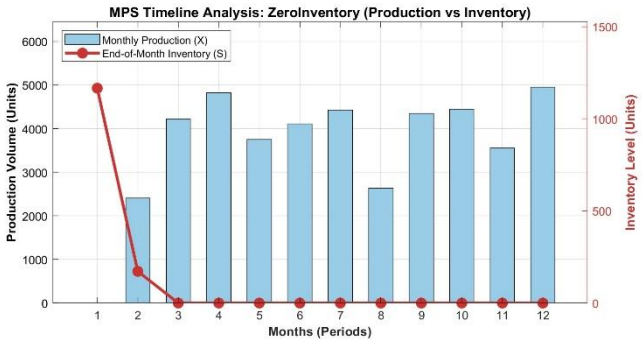
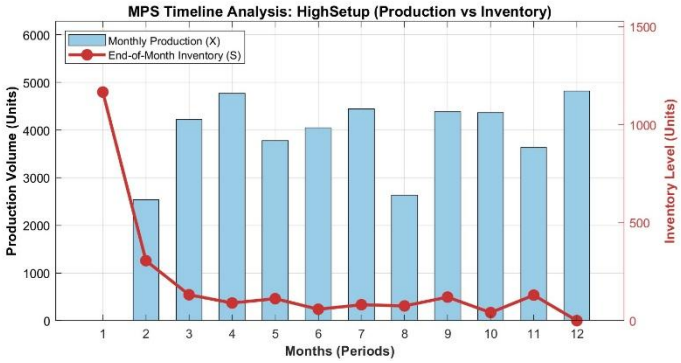
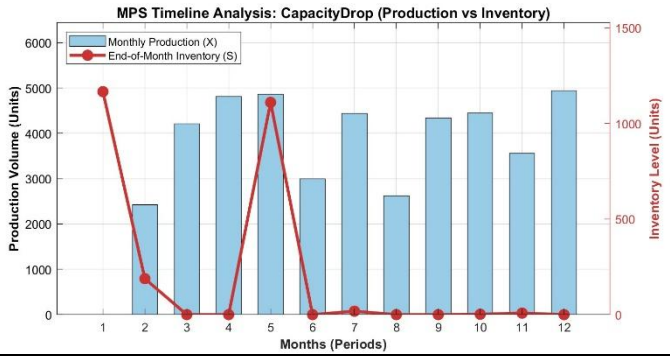
- **Scenario C: Severe Capacity Contraction:** To test the system's response to unexpected machine breakdowns or major scheduled maintenance, we halved the available capacity (`AvailCap[m,t]`) during a specific period (e.g., month 6). Rather than simply returning an unfeasible status, the model proved its ability to plan ahead. It automatically anticipated the upcoming bottleneck by shifting part of the production workload to the preceding months. By strategically building a temporary safety stock in advance, the system successfully navigated the capacity shortage and still delivered all the orders previously approved by the RRP model.

Comparisons: baseline vs model variations

A fundamental result, immediately evident from the figure, is that the Total Production Volume remains unchanged at **43,664** units across all four scenarios. This demonstrates the absolute reliability of the model: regardless of the cost penalties or capacity constraints imposed, the algorithm never degrades the Service Level, always guaranteeing 100% fulfilment of the orders **accepted at the RRP level**. The divergences between the scenarios manifest exclusively in the internal factory strategies: when to produce and how to manage inventory.



Below there is a detailed analysis of the three variations compared to the Baseline:

	An extreme artificial inflation was applied to the unit holding cost parameter, making product storage economically prohibitive. As seen in the timeline chart, the system forces the inventory level to be the lower as possible, keeping on the shelves only what is strictly unavoidable. To meet demand without using the warehouse as a buffer, the model is forced to fragment production: the number of completed batches (SPO) rises to the maximum value among all scenarios (556 batches). Financially, this hyper fragmentation combined with the penalized residual holding costs causes the total MPS cost to explode to 1.1 million €, because to force the solver to keep lower inventory we have increased the holding cost.
	A highly rigid manufacturing environment was simulated, where format changes and machine retooling are slow and expensive, increasing the setup cost parameter. Faced with massive penalties for maximize the lot size. The overall number of completed batches drops to an all time low (529). The solver launches big batches reaching the physical limit allowed by the maximum batch capacity. Consequently, a temporary overproduction is generated: the system prefers to bear a significant increase in inventory (the aggregate monthly total goes from 1382 to 2311 units) rather than shut down the machines. The total cost doubles compared to the Baseline (0.63 million €).
	To evaluate the system's resilience against prolonged breakdowns or scheduled equipment overhauls, the available production capacity (AvailCap) was artificially halved (a 50% reduction) during Period 6. This scenario shows how the model can validate the anticipation of the production in case of limited machine capacity during a period.

Chapter 4: Sustainable Material Requirement Planning (Green MRP)

4.1 Scope and Assumptions

Following the definition of the Master Production Schedule (MPS), which dictates the exact volume and timing of finished goods, the Material Requirements Planning (MRP) model determines the precise purchasing schedule for raw materials and components. **As a deliberate design choice to elevate the comprehensiveness of our project, we decided to expand beyond a standard MRP by engineering a "Green" logistics framework directly into the core system.** The calculation of CO2 emissions has become an integral, structural part of our transportation logic. Consequently, our implementation autonomously minimizes the total procurement cost while simultaneously penalizing the carbon footprint generated by the supplier network, reflecting modern sustainability challenges. To accurately capture the real-world dynamics of our model, the MRP is grounded in the following core assumptions:

- **Zero-Month Supplier Lead Time (`LT_Supp = 0`):** Since our planning horizon operates on discrete monthly intervals (`DaysInMonth = 30`), and our selected suppliers are located within a geographic radius that allows for delivery in less than 30 days, we assume the supplier lead time (`LT_Supp`) to be zero in a monthly context. An order of components launched within a specific month (t) physically arrives and is available for assembly within the very same monthly period. **This is the main assumption for this model**.
- **Rigid Packaging Constraints (`NumBoxes`):** Suppliers do not ship loose, single components. Procurement is strictly bound to pre-defined packaging capacities (`PcsPerBox` on the Access Database). Consequently, the ordered quantity of components is always an exact mathematical multiple of the number of components contained in a standard box size, forcing the system to occasionally over-order and store the excess components in the warehouse.
- **Distance-Based Carbon Taxation (`SetupOrderCost`):** The fixed cost to trigger a supplier delivery is not merely administrative. It is explicitly calculated based on the physical distance (`Distance[s]`) travelled by the truck. This cost heavily incorporates a sustainability parameter, charging a direct penalty for the calculated CO2 emissions (`CO2EmissionPerKm` and `CO2Tax`), pushing the solver to minimize the number of trips of the supplier.
- **Volumetric Holding Cost (`HoldingCost`):** Storing raw materials consumes warehouse capacity based on their mass. The assumed daily storage rate (`StorageCostPerKg`) is multiplied by 30 days to reflect the monthly time bucket. Crucially, the calculation includes not only the weight of the raw component (`UnitWeight`) but also the distributed tare weight of its packaging (`BoxWeight / PcsPerBox`).

- **Dual-Zone Warehouse Segregation:** Although the facility operates within a single physical plant, thereby eliminating internal transportation lead times and costs between storage and the shop floor, we assume a strict physical and logical segregation of inventory. The plant features an Inbound Raw Materials Stockroom (governed by the `Inv` variable and initialized via `MagInit`) and an Outbound Finished Goods Warehouse (governed by the `S` variable and initialized via `InitInv` in the MPS). To represent a highly realistic, ongoing industrial operation, the inbound stockroom does not start empty; it is accurately initialized via the `MagInit` parameter on Access to account for the residual component inventory inherited from the previous year's operations (carry-over stock). Both zones share a unified mass-based cost logic driven by the identical `StorageCostPerKg` parameter. However, this mathematical separation is essential to manage the distinct initialization pipelines and to correctly allocate the additional supplier packaging tare weights (`BoxWeight`) exclusively to the inbound raw materials.

4.2 Key Parameters and Time Phasing (Explosion of the BOM)

The fundamental link between the MPS output and the MRP input is the Bill of Materials (BOM). The `GrossReq` parameter (Gross Requirements) represents the exact mathematical "explosion" of the MPS.

The system iterates through the BOM (`nProdComp`), calculating exactly how many individual sub-components are required to manufacture the finished goods planned for a specific month.

Because the finished goods require a production lead time `LTPP`, defined in the MPS, the components must be physically present in the warehouse before the manufacturing cycle begins. The algorithm manages this temporal shift automatically through this formula:

```
param GrossReq {c in COMPONENTS, t in PERIODS} :=
    sum {(p,c) in BOM: t + LTPP[p] <= MaxT}
    (MPS_Plan[p, t + LTPP[p]] * nProdComp[p,c]);
```

Where the variable `MPS_Plan [p, t+LTPP[p]]` is the variable `X` of the MPS, being, in this case, the produced quantity of product p at time t+LTPP which is therefore ready to be sold or stocked in the inventory. The linking between MPS_Plan and X variable is defined in the file MRP.run. If the MPS dictates that a finished Brake will exit the machine in month 2, and the Brake takes 1 month to be built, the `GrossReq` for its internal components (e.g., BrakeBox) is strictly generated in month 1.

To manage this complex temporal flow, the model utilizes four main decision variables:

- `ArrivalQty[c,s,t]`: The physical volume of component `c` successfully arriving at the facility in month `t` from supplier `s`.
- `Inv[c,t]`: The physical inventory of raw component `c` remaining in stock at the end of month `t`.
- `Y[c,s,t]`: A binary logistical switch (0 or 1). If active, it indicates that the truck from supplier `s` carrying component `c` has been dispatched in month `t`.

- `NumBoxes[c,s,t]`: An integer variable determining the exact number of full boxes of component c purchased from supplier `s` at time `t`.
- `RO_Comp[c,s,t]`: The total quantity of components officially issued on the purchase order sent to supplier `s` in month `t`. Unlike the gross requirements (`GrossReq`), which reflect the exact net consumption calculated by the BOM explosion, this variable represents the actual physical volume ordered. It is strictly constrained to be an exact multiple of the supplier's packaging units (`PcsPerBox`), capturing the discrepancy between real manufacturing demand and commercial purchasing lots.

4.3 Objective Function

The primary objective of the Green MRP model is to guarantee the physical availability of raw materials required by the Master Production Schedule, while strictly minimizing the total procurement and holding costs. Furthermore, the objective function directly embeds the environmental cost of the logistics network.

The solver minimizes the sum of three distinct economic drivers:

```
#OBJECTIVE FUNCTION
minimize Total_MRP_Cost:
    sum {t in PERIODS, c in COMPONENTS}
        (HoldingCost[c] * Inv[c,t]) +
    sum {t in PERIODS, (c,s) in COMP_SUPP}
        ((CostOfBox[c,s] / PcsPerBox[c]) * ArrivalQty[c,s,t] + SetupOrderCost[s] * Y[c,s,t]);
```

• **Inventory Holding Cost:** Penalizes the accumulation of stock (`Inv[c,t]`), calculated volumetrically and by mass (including packaging tare).

• **Variable Procurement Cost:** The pure purchasing price of the physical components arriving at the facility (`ArrivalQty[c,s,t]`). As defined in the Access database, the `CostOfBox` parameter represents the all-inclusive price of a full package, encompassing both the physical packaging and the raw components contained within it. To maintain dimensional consistency and calculate the exact unit cost of a single component, this total package price is divided by the pre-defined number of pieces it contains (`PcsPerBox`).

• **Green Setup Order Cost:** This is where the environmental penalty is activated. The binary variable `Y[c,s,t]` acts as a logical switch representing the dispatch of a supplier's truck. When activated, it triggers the fixed `SetupOrderCost`, which mathematically incorporates both the standard transportation expense (`Distance · CostPerKm`) and the explicit environmental penalty (`Distance · CO2EmissionPerKm · CO2Tax`).

4.4 Constraints

To ensure the purchasing plan is both temporally accurate and physically executable by the suppliers, the optimization is bounded by a strict set of logical and logistical constraints.

- **Initial Inventory (`Init_Inventory`):** This constraint establishes the baseline state of the Inbound Raw Materials Stockroom. It anchors the inventory at period zero (`Inv[c,0]`) to the specific `MagInit[c]` parameter imported from the Access database, representing the carry-over stock already available at the facility before the year begins.

```
subject to Init_Inventory {c in COMPONENTS}:
    Inv[c, 0] = MagInit[c];
```

- **Inventory Flow Balance (`Inv_Balance`):** It is the core conservation of mass equation. It dictates that the physical raw materials carried over from the previous month (`Inv[c, t-1]`) plus the new arrivals from suppliers in the current month (`ArrivalQty[c,s,t]`) must exactly cover the production requirements generated by the BOM explosion (`GrossReq[c,t]`) plus whatever is kept in stock for the future (`Inv[c,t]`).

```
subject to Inv_Balance {c in COMPONENTS, t in PERIODS}:
    Inv[c, t-1] + sum {s in SUPPLIERS: (c,s) in COMP_SUPP} ArrivalQty[c,s,t] = GrossReq[c,t] + Inv[c,t];
```

- **Supplier Lead Time Offset (`LeadTime_Offset`):** This constraint governs the temporal shift between the order release (`RO_Comp`) and the physical receipt of goods (`ArrivalQty`). Because we assume a zero-month supplier lead time (`LT_Supp = 0`), the operational condition (`t > LT_Supp` which is 0) is strictly satisfied across the entire active planning horizon (`t = 1…12`). Consequently, the constraint is fully active and mathematically simplifies to `ArrivalQty[c,s,t]= RO_Comp[c,s,t]`. **This direct equivalence forces the component order release and its physical arrival to occur instantaneously within the exact same monthly period.**

```
#LeadTime contraint (remember in this mrp the leadtime is zero month)
subject to LeadTime_Offset {(c,s) in COMP_SUPP, t in PERIODS: t > LT_Supp[s]}:
    ArrivalQty[c,s,t] = RO_Comp[c,s, t - LT_Supp[s]];
```

- **Pipeline Initialization (`Pipeline_Init`):** This boundary condition is designed to prevent the system from receiving components before the physical transit time has elapsed. Since the supplier lead time is zero, the trigger condition ($t < 0$) is mathematically false for all active planning months. Therefore, while this specific constraint remains inactive in our current zero-lead-time scenario, it is structurally maintained within the code to ensure the model's complete scalability, allowing it to seamlessly handle distant suppliers with longer lead times in future iterations.

```
subject to Pipeline_Init {(c,s) in COMP_SUPP, t in PERIODS: t <= LT_Supp[s]}:
    ArrivalQty[c,s,t] = 0;
```

- **Rigid Packaging Multiplier (`Box_Multiplier`):** This constraint enforces the real-world limitation that suppliers do not ship loose items. It forces the total released order quantity (`RO_Comp[c,s,t]`) to be an exact multiple of the standard box capacity (`PcsPerBox[c]`), governed by the integer decision variable `NumBoxes[c,s,t]`.

```
subject to Box_Multiplier {(c,s) in COMP_SUPP, t in PERIODS}:
    RO_Comp[c,s,t] = PcsPerBox[c] * NumBoxes[c,s,t];
```

- **Truck Dispatch Switch via Big-M (`Supplier_Selection`):** To correctly charge the logistics and CO2 penalties, we implemented a Big-M formulation. If the solver decides to order even a single box (`NumBoxes > 1`), the binary switch `Y` is mathematically forced to 1. This instantly triggers the `SetupOrderCost` in the objective function, heavily penalizing fragmented, high-frequency deliveries.

```
subject to Supplier_Selection {(c,s) in COMP_SUPP, t in PERIODS}:
    NumBoxes[c,s,t] <= BigM * Y[c,s,t];
```

- **Minimum Order Lot (`Min_Order_Lot`):** This logical boundary (`NumBoxes[c,s,t]` ≥ `Y[c,s,t]`) prevents the solver from exploiting mathematical loopholes. It ensures that if the truck order is active (`Y=1`), the model must order at least one full physical box, preventing scenarios where transportation and environmental costs are paid for empty deliveries.

```
subject to Min_Order_Lot {(c,s) in COMP_SUPP, t in PERIODS}:
    NumBoxes[c,s,t] >= Y[c,s,t];
```

4.5 Results Analysis and Model Verification

The execution of the Green MRP model successfully concludes the hierarchical optimization framework. By translating the operational schedule (`MPS_Plan`) into precise material requirements, the Gurobi solver generated a strictly feasible purchasing plan that manages supplier distances, physical packaging limits, and environmental sustainability constraints. The optimization model reached a minimum, yielding a total supply chain expense of **€9,752,840** (`Total_MRP_Cost`). This economic figure represents the absolute optimal balance between three conflicting logistical drivers: the capital tied up in the warehouse (`HoldingCostComp`), the actual purchasing cost of the boxed components (`CostOfBox`), and the fixed delivery penalties (`SetupOrderCost`) which integrate both transport distances and the applied CO2 emission taxes.

```
Total_MRP_Cost = 9752840
```

4.5.1 Mass-Balance and Safety Stock Absorption

To verify the core flow equation (`Inv_Balance`), we analyse the `BrakeBox` component. Because the raw materials are bound by fixed packaging multiples, the solver is forced to order more physical units than the exact theoretical `GrossReq` requires, mathematically resulting in residual inventory.

- At **t = 0**, the system successfully registers the initial inventory parameter (`MagInit = 100` units).
- In **January (t = 1)**, the solver decides to replenish the stock by purchasing 122 full boxes from Supplier S30 (`NumBoxes['BrakeBox', 'S30', 1] = 122`). Assuming each box contains exactly 50 pieces (`PcsPerBox[c] = 50` in the Access Database), this triggers a physical inbound flow of 6100 components (`ArrivalQty['BrakeBox', 'S30', 1] = 6100`).

```
NumBoxes [*,*,1]
:              S01   S02 S(
Bearing          .     .
Billet           .     .
BrakeBox         .     .
BrakeDisc        0     0
ClutchBox        .     .
ClutchCover      0     .
ClutchDisc       .     0
ConnectingRod    .   212
Grip             .     0
Head             .     .
Nuts             .     0
PistonCase       .     0
PreassurePlate   0     .
RawClutch        .     .
Seal             .     .
ShaftCase        .     0
ShaftRaw         0     .
Sheet            .     .
ShiftCase        0     .
ShiftRaw         .     .
WheelContainer   .   114
WheelMaterial    .     .

:              S30    :=
BrakeBox       122
ClutchBox        0
RawClutch        0
WheelContainer   0
```

```
ArrivalQty [*,*,1]
:              S01   S02  S03
Bearing          .     .    .
Billet           .     .    .
BrakeBox         .     .    .
BrakeDisc        0     0    0
ClutchBox        .     .    .
ClutchCover      0     .    0
ClutchDisc       .     0    .
ConnectingRod    .   212    0
Grip             .     0    .
Head             .     .    0
Nuts             .     0    .
PistonCase       .     0    .
PreassurePlate   0     .    .
RawClutch        .     .    .
Seal             .     .    0
ShaftCase        .     0    0
ShaftRaw         0     .    .
Sheet            .     .    0
ShiftCase        0     .    .
ShiftRaw         .     .    0
WheelContainer   .  3420    0
WheelMaterial    .     .    .

:              S28 S29   S30
Bearing          0   0     .
BrakeBox         .   .  6100
ClutchBox        .   .     0
Grip             .   0     .
Nuts             .   0     .
RawClutch        .   0     0
WheelContainer   0   0     0
WheelMaterial    .   0     .
```

- While the exact internal demand generated by the BOM explosion required precisely 822 units (`GrossReq['BrakeBox', 1] = 822`), the solver mathematically absorbs the commercial discrepancy of the rigid packaging into the `Inv` variable. By applying the fundamental mass-balance equation (`Inv[0] + ArrivalQty[1] - GrossReq[1]`), the system calculates 100 + 6100 - 822, resulting in an exact residual physical inventory of 5378 units at the end of Month 1 (`Inv['BrakeBox', 1] = 5378`).

```
GrossReq [*,*]
:                 1
Bearing           0
Billet         1233
BrakeBox        822
BrakeDisc      1644
ClutchBox         0
ClutchCover       0
ClutchDisc        0
ConnectingRod   312
Grip           1644
Head            624
Nuts          32120
PistonCase      312
PreassurePlate    0
RawBrake          0
RawClutch         0
Seal            936
ShaftCase      1965
ShaftRaw        655
Sheet             0
ShiftCase         0
ShiftRaw          0
WheelContainer  352
WheelMaterial  1408
```

```
Inv [*,*]
:                 0      1
Bearing         100    100
Billet          100     17
BrakeBox        100   5378
BrakeDisc       100      0
ClutchBox       100    100
ClutchCover     100    100
ClutchDisc      100    100
ConnectingRod   100      0
Grip            100      0
Head            100      0
Nuts            100    230
PistonCase      100      4
PreassurePlate  100    100
RawBrake        100    100
RawClutch       100    100
Seal            100  15264
ShaftCase       100      5
ShaftRaw        100      1
Sheet           100    100
ShiftCase       100    100
ShiftRaw        100    100
WheelContainer  100   3168
WheelMaterial   100      2
```

- Crucially, the model demonstrates perfect end-of-horizon lean behaviour. From Month 10 to Month 12, the `BrakeBox` inventory drops progressively from 1486 down to just 24 units, proving the algorithm avoids unnecessary purchasing at the end of the year, strictly minimizing the holding cost penalty.

```
Inv [*,*]
:                 0      1       2       3       4       5       6
Bearing         100    100       4       0       8       4       8
Billet          100     17      48       8      39      41      19
BrakeBox        100   5378    4432    3572    2626    1594     646
BrakeDisc       100      0       0       0       0       0    1200
ClutchBox       100    100       3       0       1       3     576
ClutchCover     100    100       0       0       0       0     144
ClutchDisc      100    100       0       0       0       0       0
ConnectingRod   100      0       0       0     486       0     324
Grip            100      0       0       0       0       0       0
Head            100      0       0       0       0       0       0
Nuts            100    230     130      92      86      82     196
PistonCase      100      4       0       0     486       0     326
PreassurePlate  100    100       0     576       0     432     144
RawBrake        100    100     100     100     100     100     100
RawClutch       100    100     100     100     100     100     100
Seal            100  15264   12348   12348   11214    9756    7974
ShaftCase       100      5       5     710       2       7       7
ShaftRaw        100      1       1       0       0       1       1
Sheet           100    100    24.8       2     5.6     0.8     5.6
ShiftCase       100    100       4       2       8       0       6
ShiftRaw        100    100       0       0       0       0       0
WheelContainer  100   3168    2592    1368     504       0    1734
WheelMaterial   100      2       3       2       1       0       1

:                 7      8       9      10      11      12      :=
Bearing           0      2       0       6       6       6
Billet           19     34       4       4      11      11
BrakeBox         46   3206    2086    1486      24      24
```

4.5.2 The "Green" Logistical Switch

The true innovation of the model lies in its integration of the Big-M constraint (`Supplier_Selection`), which ties the physical purchase of boxes to the binary dispatch of a truck (`Y[c,s,t]`), instantly triggering the `SetupOrderCost` (which structurally includes the CO2 emission penalty). We can mathematically verify this mechanism by analysing the `BrakeBox` component sourced from Supplier S30.

- In **Month 1**, the model determines it needs exactly 122 full boxes (`NumBoxes['BrakeBox', 'S30', 1] = 122`). Since 122 > 0, the Big-M equation mathematically forces the binary variable to activate (`Y['BrakeBox', 'S30', 1] = 1`). This exact activation correctly charges the objective function with the specific logistical and environmental penalty for S30, which amounts to €896.10.
- By contrast, in **Month 2**, the internal inventory is sufficient, and no boxes are required from this specific supplier (`NumBoxes['BrakeBox', 'S30', 2] = 0`). The mathematical constraints correctly drop the binary switch to zero (`Y['BrakeBox', 'S30', 2] = 0`), preventing any phantom emission charges and proving that the supply chain is optimized purely on a Just-In-Time transportation logic.

```
NumBoxes [*,*,1]
:              S01   :
Bearing          .
Billet           .
BrakeBox         .
BrakeDisc        0
ClutchBox        .
ClutchCover      0
ClutchDisc       .
ConnectingRod    .
Grip             .
Head             .
Nuts             .
PistonCase       .
PreassurePlate   0
RawClutch        .
Seal             .
ShaftCase        .
ShaftRaw         0
Sheet            .
ShiftCase        0
ShiftRaw         .
WheelContainer   .
WheelMaterial    .

:              S30
BrakeBox       122
```

```
 [*,*,2]
:              S01   S02
Bearing          .     .
Billet           .     .
BrakeBox         .     .
BrakeDisc        0     0
ClutchBox        .     .
ClutchCover     88     .
ClutchDisc       .     0
ConnectingRod    .   972
Grip             .     0
Head             .     .
Nuts             .     0
PistonCase       .     0
PreassurePlate   0     .
RawClutch        .     .
Seal             .     .
ShaftCase        .     0
ShaftRaw         0     .
Sheet            .     .
ShiftCase       75     .
ShiftRaw         .     .
WheelContainer   .     0
WheelMaterial    .     .

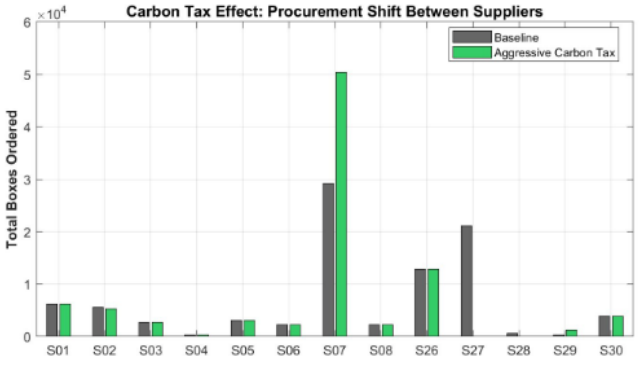
:              S29   S30
Bearing         28     .
BrakeBox         .     0
```

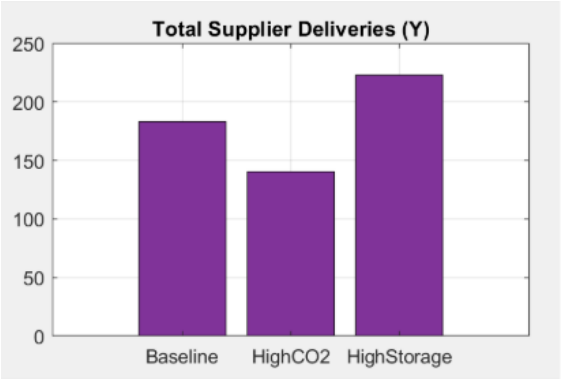
```
SetupOrderCost [*] :=
S01   208.575
S02   618
S03   355.35
S04   865.2
S05    38.625
S06   105.06
S07    83.43
S08   463.5
S26   324.45
S27  1096.95
S28  1328.7
S29   412.515
S30   896.1
;
```

This absolute synchronization between the integer box quantities and the binary logistical triggers confirms the total reliability of the sustainable procurement logic.

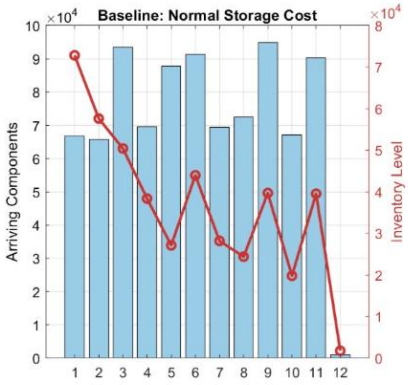
4.6 Model Validation and Stress-Testing

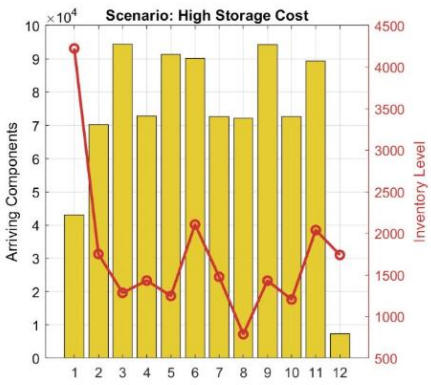
- **Scenario A: Aggressive Carbon Taxation:** To evaluate the sensitivity of the supply chain to sudden environmental legislation, we simulated an aggressive regulatory shock by drastically increasing the `CO2Tax` parameter. Prior to this changing, Gurobi naturally favoured geographically distant suppliers, strictly due to their lower initial component price (`CostOfBox`). However, as the carbon tax surged, the logistical penalty (`SetupOrderCost`) for these suppliers became economically prohibitive due to their high transit emissions. The decision support system autonomously re-routed the procurement matrix, shifting the purchased volume (`NumBoxes[c,s,t]`) toward a geographically closer, slightly more expensive, but highly emission-efficient supplier. This is the proof that the system is capable to balance standard financial margins against strict environmental, social, and governance targets.
- **Scenario B: Volumetric Storage Cost Surge:** To stress-test the inventory holding logic, we simulated a severe operational environment where the introduction of specialized, exceptionally heavy, and bulky mechatronic assemblies rapidly saturates the baseline warehouse capacity. To safely handle and store these oversized components, the facility would be forced to utilize specialized heavy-duty racking systems or rent premium overflow storage space. We mathematically mirrored this physical constraint by artificially inflating the baseline `StorageCostPerKg` parameter since we do not have, as assumed from the beginning, a parameter for representing the warehouse maximum capacity in terms of space. Faced with these exorbitant holding penalties for heavy components, the MRP framework was mathematically forced into an extreme Just-In-Time (JIT) procurement pattern. To systematically avoid holding raw materials in stock (`Inv[c,t]`) the solver dismantled consolidated bulk orders. It intentionally increased the frequency of supplier deliveries, repeatedly activating the binary logistical triggers (`Y[c,s,t]=1`). The system deliberately chose to absorb a significant increase in transport and CO2 penalties to strictly prevent any inventory accumulation, confirming the robust internal trade-off mechanics between inbound logistics and warehouse management.





Scenario A: Once the high carbon tax is applied, the logistical penalty for highly emissive transport routes becomes economically prohibitive. As illustrated in the supplier procurement graph, the Decision Support System (DSS) autonomously reacts by shifting the purchased volumes toward suppliers with low carbon emissions (S07 which is only 54 Km far away from our company), effectively decreasing the deliveries from highly emissive suppliers. For example, the deliveries from the supplier S27, that is 710 Km far away from our company, decreased to zero in the case of the aggressive carbon tax.





Scenario B: Faced with exorbitant holding penalties, the MRP framework was forced into an extreme Just-In-Time (JIT) procurement pattern. As illustrated in the Scenario B graph, the system intentionally increases the frequency of supplier deliveries, repeatedly activating logistical triggers to procure exact quantities precisely when required. This dynamic effectively drives standing inventory levels to a strict minimum. The solver deliberately chose to absorb a significant surge in transport operations, executing over 220 deliveries compared to the 180 of the baseline case, to strictly prevent any inventory accumulation.

Chapter 5: Operational Job Shop Scheduling

5.1 Scope and Physical Assumptions

While the RRP calculated the aggregate long-term resource capacity and the MPS defined the monthly production volumes, the execution on the shop floor occurs at the **minute level**. The Job Shop Scheduling module is a high-resolution, short-term operational framework. It translates the mathematical matrices of the upper-level plans into a strictly sequential timeline of operations, definitively answering two questions: ***when* each individual batch begins processing, *which exact* physical machine executes the work and *how much* is the total makespan.**

Because this model dictates the precise physical movements within the facility, it requires a rigid set of operational assumptions:

- **Precedence Network and Handling Times (`Handling`):** The manufacturing environment is not a free-flow system. Every product family follows a strict, immutable technological routing (`Precedente` on the Access Database). Crucially, **operations on a specific batch cannot begin on a subsequent machine until the processing on the current machine is entirely completed and the entire batch has been physically transported**. This transit time is natively calculated by the `Handling` parameter managed in the proper constraint, which absorbs the inter-machine logistics.
- **Parallel Machine Topology (`MachQuantity` & `Alloc`):** To resolve process bottlenecks, the facility operates multiple distinct physical copies of the same machine type. The solver possesses the spatial awareness to autonomously route competing batches across these parallel resources. Crucially, the available quantity for automated CNC machinery is strictly bound to the exact purchasing plan finalized in the Resource Requirements Planning (RRP) module. **However, an explicit internal management decision was made regarding the manual assembly phase**. Because the `AssemblyTables` constitute pre-existing internal infrastructure rather than capital investments, they were deliberately excluded from the RRP and MPS capital expenditure calculations (defaulting to zero in the purchasing output on the Access Database). **To accurately reflect the physical shop floor capacity for the scheduling phase, the Access database must be now manually updated to declare 13 pre-existing, fully operational `AssemblyTables`, establishing the absolute highest degree of simultaneous parallelism within the facility**. This architectural choice is driven by the physical workflow: while heavy automated machinery rarely exceeds 3 or 4 parallel units due to high capital expenditure, the 13 assembly benches must operate almost entirely simultaneously, as every single mechatronic component exiting the processing phase ultimately requires manual assembly from the operator.
- **Asymmetric Labor Allocation (`WorkersPerBench`):** Based on the overall workforce parameters established in the RRP (61 total workers), we divided the labour pool into two distinct teams. The first group is composed by **22 highly mobile workers acting as an *internal logistics and setup team*, floating between heavy automated CNC machinery** (where `WorkersPerBench = 1`) and this is a realistic choice because from the RRP we purchased 16 machines, and this means having an average of 1.4 operators per machine. Conversely, the final assembly phase relies purely on manual dexterity. Therefore, **the**

remaining 39 workers are distributed across 13 parallel manual `AssemblyTables`**, permanently assigning exactly 3 operators per bench** (`WorkersPerBench = 3`). This asymmetric modelling drastically reduces the makespan by accelerating the manual processing phase without inflating the workforce overhead.

- **Batch Disaggregation and `Jobs` Database Table:** To manage the scheduling at a highly granular operational level, the aggregate monthly production volumes calculated by the MPS cannot be processed as a single continuous flow. They must be explicitly disaggregated into distinct, executable batches. This structural requirement necessitated the creation of a new, dedicated Access database table named `Jobs`. Within this table, the total production requirement for every product family is divided into specific, discrete lots. Each lot is uniquely identified by a sequential integer number (`LotN`) and is assigned a strictly defined lot-size parameter (`Quantity`). This architectural setup is the backbone of the scheduling framework, allowing the algorithm to independently trace, sequence, and route individual physical batches across the shop floor.

- **Non-Preemptive Batch Processing (`Quantity`):** Once a specific machine initiates the setup and begins processing a defined lot (`p,i`), the operation cannot be interrupted, paused, or fragmented. The machine remains entirely locked until all pieces within the `Quantity` parameter have been processed.

5.2 Mathematical Variables and Decision Architecture

To prevent combinatorial explosion and ensure computational feasibility within Gurobi, the model relies on a highly efficient combination of continuous tracking variables and binary logic switches.

- `Makespan`: A continuous, non-negative variable acting as the universal clock of the shop floor. It measures the absolute total time (**in minutes**) required to process, transport, and assemble every single batch released into the system. Therefore, it delivers the exact total manufacturing duration required to fulfill the complete sequence of production lots manually configured and inserted by the user within the Access database (`Jobs` table).
- `StartTime[p,i,m]`: The fundamental scheduling variable. It marks the exact integer minute when batch i of product p begins its setup phase on a specific machine type m.
- `Alloc[p,i,m,k]`: A sophisticated spatial binary switch. It physicalizes the schedule. If this variable triggers to 1, the specific batch is unequivocally assigned to the physical copy k of machine m. The search space is bounded by the exact `MachQuantity` parameter derived from the RRP investment plan.
- `Z[p1,i1,p2,i2,m]`: The sequence binary switch. This is the core of the disjunctive programming framework. When two independent batches require the same machine type, this variable determines the queue order. A value of 0 grants priority to the first batch, while a 1 reverses the order. To optimize the solver's branching speed, this variable is only generated when strictly necessary (e.g., ord(p1) < ord(p2)), systematically eliminating redundant mirrored comparisons (e.g. Brake vs Clutch and Clutch vs Brake).

5.3 Objective Function

The scheduling framework operates under a strictly defined min-max objective function. The primary goal is to minimize the total makespan, represented by the continuous variable `Makespan`. Mathematically, this forces the Gurobi solver to compress the entire sequence of batch operations into the shortest possible absolute time horizon, dynamically optimizing the temporal density of the shop floor and maximizing the overall plant throughput.

```
# OBJECTIVE FUNCTION
minimize C_max:
    Makespan;
```

5.4 Mathematical Constraints

The physical and temporal boundaries of the manufacturing facility are enforced through a strict system of linear and disjunctive constraints:

- **Machine Allocation Bound (`Assign_To_One_Copy`):** To enforce physical reality, a single, non-preemptive batch cannot be splitted across multiple machines to artificially reduce processing time, nor can it bypass a required technological step. This constraint mandates that the sum of the binary allocation variables for a specific lot on a required machine type m must strictly equal 1 across all available machine physical copies (`k`).

```
subject to Assign_To_One_Copy {(p,i) in JOBS, m in MACHINES: (p,m) in PRODMACH}:
    sum {k in COPIES: k <= MachQuantity[m]} Alloc[p,i,m,k] = 1;
```

- **Makespan Definition Bound (`Calc_Makespan`):** This is the objective variable, and it is bound to track the global completion time. This constraint guarantees that the overall Makespan is greater than or equal to the final completion time of every single operation. The completion time of a batch is calculated as its start time, plus the machine setup, plus the total active processing time. Crucially, the pure operational duration is dynamically adjusted by the workforce density parameter (`WorkersPerBench[m]`). For standard technological machinery (e.g., drilling, welding, or lathing), exactly 1 operator is assigned per machine (`WorkersPerBench = 1`), leaving the nominal processing time unchanged; this single operator is responsible for the machine setup and for the physical handling of the batch to the subsequent phase. Conversely, for the `AssemblyTable`, the parameter is set to 3 (`WorkersPerBench = 3`). We set the `ProcTime` for the `AssemblyTable` in the Access database as the pure `ManualAssembly` (time) of the products, then dividing this baseline parameter by 3 mathematically collapses the overall duration, perfectly simulating the accelerated throughput generated by three operators working simultaneously on the final manual consolidation. Furthermore, to reflect actual shop floor dynamics, the `SetupTime` for the `AssemblyTable` was explicitly set to zero. Unlike automated machinery that requires mechanical tooling and calibration, manual assembly benches demand no complex preparatory setup; the process can begin the exact moment the components physically

arrive at the workstation. These changes have been made in the PRODMACH table in the Access Database. **It is important to clarify that the logistical transit delay of a product from one machine to the other does not mathematically appear in this specific equation; instead, its temporal impact is explicitly managed by the subsequent Intra-Job Routing constraint, which automatically pushes the `StartTime` forward to absorb the `Handling` (time) parameter.**

```
subject to Calc_Makespan {(p,i) in JOBS, (p,m) in PRODMACH}:
    Makespan >= StartTime[p,i,m] + SetupTime[p,m] + ((ProcTime[p,m] * Quantity[p,i]) / WorkersPerBench[m]);
```

- **Intra-Job Routing and Handling (`Intra_Job_Routing`):** This constraint maps the chronological assembly line defined by the Bill of Routings. It mathematically prevents an operation on machine `m` from commencing until the batch has completely finished processing on its exact preceding machine (`Precedente`) ***and*** has physically traversed the facility to reach the actual machine `m`. The spatial transit delay is natively integrated via the `Handling[p]` parameter.

```
subject to Intra_Job_Routing {(p,i) in JOBS, (p,m) in PRODMACH:
    Precedente[p,m] != "" and Precedente[p,m] != "Nessuno"}:
    StartTime[p,i,m] >= StartTime[p,i,Precedente[p,m]] + SetupTime[p,Precedente[p,m]] +
    ((ProcTime[p,Precedente[p,m]] * Quantity[p,i]) / WorkersPerBench[Precedente[p,m]]) + Handling[p];
```

- **Disjunctive Non-Overlap Logic (`No_Overlap_A` & `No_Overlap_B`):** This is the computational core of the scheduling, leveraging a **Big-M formulation** to prevent physical collisions. When two distinct batches compete for the exact same machine type `m`, they can only run simultaneously if the solver assigns them to different physical copies (e.g., `Alloc_1 = 1` on copy 1, and `Alloc_2 = 1` on copy 2). However, if they are routed to the exact same physical copy, the binary switch `Z` triggers to separate them chronologically. The Big-M forces one of two mutually exclusive scenarios:

```
subject to No_Overlap_A { (p1,i1) in JOBS, (p2,i2) in JOBS, m in MACHINES, k in COPIES:
    (ord(p1, PRODUCTS) < ord(p2, PRODUCTS) or (p1 == p2 and i1 < i2))
    and (p1,m) in PRODMACH and (p2,m) in PRODMACH and k <= MachQuantity[m]}:
    StartTime[p2,i2,m] >= StartTime[p1,i1,m] + SetupTime[p1,m] + ((ProcTime[p1,m] * Quantity[p1,i1]) / WorkersPerBench[m])
                        - BigM * (1 - Z[p1,i1,p2,i2,m])
                        - BigM * (2 - Alloc[p1,i1,m,k] - Alloc[p2,i2,m,k]);

subject to No_Overlap_B { (p1,i1) in JOBS, (p2,i2) in JOBS, m in MACHINES, k in COPIES:
    (ord(p1, PRODUCTS) < ord(p2, PRODUCTS) or (p1 == p2 and i1 < i2))
    and (p1,m) in PRODMACH and (p2,m) in PRODMACH and k <= MachQuantity[m]}:
    StartTime[p1,i1,m] >= StartTime[p2,i2,m] + SetupTime[p2,m] + ((ProcTime[p2,m] * Quantity[p2,i2]) / WorkersPerBench[m])
                        - BigM * Z[p1,i1,p2,i2,m]
                        - BigM * (2 - Alloc[p1,i1,m,k] - Alloc[p2,i2,m,k]);
```

If the batches share the same copy, (`2 - Alloc_1 - Alloc_2`) becomes 0, neutralizing that relaxation term. The solver must then choose `Z=0` (granting priority to batch 1) or `Z=1` (granting priority to batch 2), effectively eliminating any temporal overlap.

5.5 Model Execution and Results Analysis

The execution of the Job Shop Scheduling model translates the monthly macroscopic volumes and tactical capacity constraints into a deterministic, minute-by-minute operational timetable. By processing the exact number of products in each single batch (`Quantity`) through the strict technological routings (`Precedente`), the Gurobi solver successfully organized the intricate puzzle of parallel machine allocations and operator assignments without any temporal overlap.

The solver achieved an optimal min-max objective value of **Makespan = 6312 minutes**.

```
Makespan = 6312
```

5.5.1 Operational Translation and Shift Feasibility

Translating this mathematical output into operational terms, the entire disaggregated workload requires exactly 105.2 hours of active manufacturing time (6312 minutes). Given the facility's heterogeneous operating structure, where certain machines (M02, M18, M20) run a single 8-hour daily shift, while others (M01, M03, M04, M05, M19, M21) operate on a double 8-hour shift, this volume represents approximately 13.15 standard shift equivalents. Even in the most conservative scenario, where the timeline is strictly bottlenecked by the single-shift machinery (requiring just over 13 calendar days to complete), the total duration easily fits within the designated monthly production bucket. This confirms that the facility possesses ample capacity to absorb the required demand without generating structural backlogs.

5.5.2 Technological Routing Validation (StartTime)

The `StartTime` output array provides empirical proof that the `Intra_Job_Routing` constraint successfully enforced the technological sequences. By tracing a highly complex physical batch, such as **Clutch, Lot 1**, we can observe a strictly monotonic temporal progression across the shop floor:

- **M01:** Starts at minute 125.
- **M02:** Starts at minute 876.
- **M03:** Starts at minute 1617.
- **M05:** Starts at minute 2207.
- **M18:** Starts at minute 3539.
- **M20:** Starts at minute 3897.
- **AssemblyTable:** Starts at minute 5772 (this is always the final operation).

```
StartTime [*,1,*] (tr)
:              Brake Clutch Piston  Shaft  Shift  Wheel    :=
AssemblyTable   6112   5772   5412   5112   3812   1731
M01              260    125   3562    575    135      .
M02                .    876   4360      .      .      0
M03             1774   1617      .   2037    869      .
M04             3292      .   4680      .   1883    406
M05                .   2207      .   2808      .    784
M18                .   3539      .   4411   2402      .
M19                .      .      .      .      .   1532
M20             3938   3897      .   4609      .      .
M21                .      .      .      .   3051      .
```

There is no point where the subsequent operation begins before the preceding one is fully completed. Furthermore, the temporal gaps between the start times (e.g., the 751-minute gap between M01 and M02) correctly absorb the batch `ProcTime`, the machine `SetupTime`, and the explicit spatial transit delays natively coded in the `Handling` parameter.

5.5.3 Parallel Allocation and Spatial Integrity (Alloc)

The output of the binary spatial variable `Alloc[p,i,m,k]` confirms the absolute integrity of the `Assign_To_One_Copy` constraint. Analysing the assignment for **Clutch, Lot 1**, the solver correctly bypassed redundant resources to prevent physical "cloning" of the batch, while fully exploiting the facility's parallel architecture.

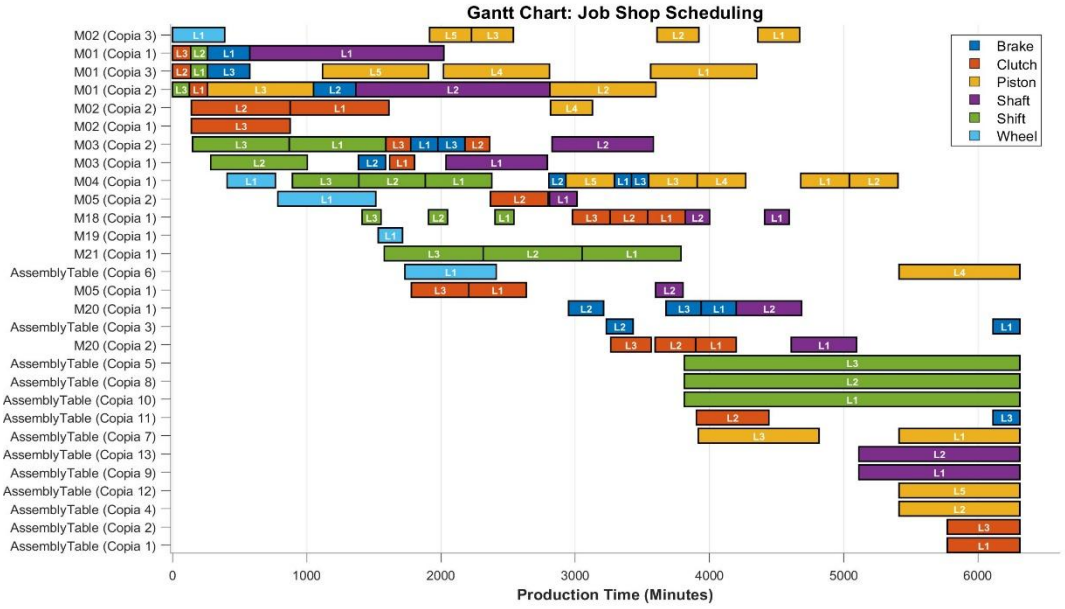
For instance, the routing for this specific lot dynamically shifts across different physical machine copies**: it is assigned to Copy 1 for machines M03, M05, M18,** and the **final AssemblyTable** (`Alloc['Clutch', 1, 'AssemblyTable', 1] = 1`), but the solver explicitly routed it to **Copy 2 for machines M01, M02,** and **M20** (`Alloc['Clutch', 1, 'M02', 2] = 1`).

[Clutch,1,*,*] (tr)

:	AssemblyTable	M01	M02	M03	M05	M18	M20	:=
1	1	0	0	1	1	1	0	
2	0	1	1	0	0	.	1	
3	0	0	0	.	.	.	.	
4	0	.	.	.	.	.	.	
5	0	.	.	.	.	.	.	
6	0	.	.	.	.	.	.	
7	0	.	.	.	.	.	.	
8	0	.	.	.	.	.	.	
9	0	.	.	.	.	.	.	
10	0	.	.	.	.	.	.	
11	0	.	.	.	.	.	.	
12	0	.	.	.	.	.	.	
13	0	.	.	.	.	.	.	

Crucially, the matrix explicitly locked all other available copies for that specific operation at exactly 0. This binary behaviour confirms that the model successfully deployed dynamic parallel routing to distribute the workload and avoid shop floor bottlenecks, without ever violating the fundamental laws of mass conservation and resource allocation capacity defined in the Access database.

5.6 Scheduling Gantt Chart



Chapter 6: Project Conclusion and Framework Synthesis

6.1 An Integrated Hierarchical Architecture

This project successfully developed and validated a fully integrated hierarchical production planning framework, spanning from strategic capacity investments to minute-level shop floor execution. By leveraging the computational power of the Gurobi solver through AMPL mathematical programming and maintaining a centralized data architecture via a Microsoft Access database, the system demonstrated how complex, multi-stage manufacturing environments can be autonomously optimized.

6.2 Strategic Capacity and Master Scheduling (RRP & MPS)

The foundation of the framework was established by the Resource Requirements Planning (RRP) module. By analysing aggregate long-term forecasts, the RRP generated a capital expenditure plan, identifying the exact quantity of automated machinery required to prevent systemic bottlenecks while autonomously rejecting unfeasible demand peaks to protect the facility's throughput. Crucially, this strategic implementation recognized existing facility assets, correctly excluding the manual AssemblyTables from the purchasing matrix. With the physical capacity locked and a strictly feasible order portfolio established, the Master Production Schedule (MPS) translated these constraints into a medium-term blueprint. The MPS formulated a multi-period environment to define exact monthly production volumes. A key achievement of this phase was the precise handling of diverse component lead times, specifically accommodating the zero-month lead time dynamics for highly responsive components (e.g., the "Shift" component in our specific case), ensuring that production logic perfectly matched physical lead-time realities.

6.3 Sustainable Material Sourcing (Green MRP)

The output of the MPS served as the direct demand input (`GrossReq`) for the **Material Requirements Planning (MRP)** phase. This model evolved beyond standard inventory tracking by introducing a "Green MRP" framework. Operating within a rigorously defined dual-zone warehouse setup (separating component carry-over stock from final product inventory, managed in the MPS), the algorithm balanced exact mass-conservation laws with rigid supplier packaging multiples.

The true innovation of this model was the implementation of environmental penalization via Big-M constraints. By coupling the integer lot-sizing decisions with binary logistical switches, the solver autonomously penalized the objective function with carbon tax charges and transport costs every time a physical truck dispatch was triggered, proving that economic and environmental sustainability can be co-optimized.

6.4 Autonomous Shop-Floor Execution (Job Shop Scheduling)

The final validation of the framework occurred at the micro-operational level. To bridge the gap between monthly aggregate volumes and daily shop floor reality, the MPS output was granularized into a dedicated `Jobs` database table. The **Job Shop Scheduling** tackled the challenge of routing discrete batches across the facility.

By deploying sophisticated disjunctive programming, utilizing sequence binary switches to prevent spatial collisions, the solver successfully mapped the chronological flow of operations. The model demonstrated advanced spatial awareness by dynamically routing batches across parallel copies of machinery (as defined by the RRP) and accelerating the final consolidation phase by allocating three operators strictly to the 13 simultaneous `AssemblyTables`. The resulting optimal makespan confirmed that the theoretical monthly plans are mathematically and physically executable within standard working shifts.

6.5 Final Remarks

In conclusion, this project delivers a robust, scalable, and highly automated decision-support system. It successfully bridges the macro-economic goals of asset management and sustainability with the micro-engineering realities of machine setup times, disjunctive routing, and asymmetric labour distribution, representing a comprehensive solution for modern autonomous manufacturing systems.